Sommaire

Livret R — Manuel Personnel **7**

Sommaire . . . 7

PARTIE I — FONDAMENTAUX R **8**

0. Installation et premiers concepts . . . 8

Installer R et RStudio . . . 8

L'interface RStudio (4 panneaux) . . . 9

Créer un script . . . 9

Exécuter du code . . . 9

Concepts fondamentaux . . . 9

1. Les bases . . . 10

Console vs Script . . . 10

Types de données . . . 10

Variables . . . 10

Vecteurs . . . 10

Fonctions de base . . . 10

Opérations logiques sur vecteurs . . . 11

Nettoyer l'environnement . . . 11

2. Data frames . . . 11

Accéder à une colonne . . . 11

Explorer un data frame . . . 11

Factor — variables catégorielles . . . 12

3. dplyr — Manipuler les données . . . 12

Les 5 verbes essentiels . . . 12

Le pipe `|>` . . . 12

group_by() + summarise() . . . 13

rename() — renommer des colonnes . . . 13

separate() — séparer une colonne en deux . . . 13

drop_na() / replace_na() — gérer les NA . . . 14

slice_min() / slice_max() — trouver les extrêmes . . . 14

Enchaîner summarise + slice — top/flop par groupe . . . 14

arrange(desc()) — tri décroissant . . . 14

count() — tableaux de fréquence . . . 14

distinct() — supprimer les doublons . . . 14

Détecter les valeurs aberrantes (outliers) . . . 15
case_when() — créer des catégories . . . 15
ifelse() — version rapide pour 2 options . . . 15
4. Données manquantes (NA) . . . 15
Le problème . . . 16
La solution : na.rm = TRUE . . . 16
Compter les NA . . . 16
NA dans group_by + summarise . . . 16
summary() vs summarise() . . . 16
5. Fichiers CSV et répertoire de travail . . . 16
CSV (Comma-Separated Values) . . . 16
Répertoire de travail . . . 17
Projets RStudio (.Rproj) — la bonne pratique . . . 17
6. ggplot2 — Graphiques . . . 18
Structure de base . . . 18
Les graphiques de base . . . 18
Graphiques avancés . . . 19
Couleur fixe vs couleur par variable . . . 20
Facettes — sous-panneaux . . . 20
Thèmes — apparence du graphique . . . 21
Ordonner les barres par valeur (pas par alphabet) . . . 21
Quand utiliser quel graphique ? . . . 21
7. R Markdown — Rapports reproductibles . . . 22
Créer un R Markdown . . . 22
En-tête basique . . . 22
En-tête avancé (rapport de thèse) . . . 22
Blocs de code (chunks) . . . 23
Bloc setup — charger packages et données (invisible) . . . 23
Options des chunks . . . 23
Options globales vs locales . . . 23
Sections . . . 24
Exemple de rapport complet . . . 24
Important . . . 25
8. Jointures de tables . . . 25
Les 3 jointures essentielles . . . 25

Exemple concret . . . 25
9. Dates avec lubridate . . . 26
Charger lubridate . . . 26
Convertir du texte en date . . . 26
Extraire des composants . . . 26
Calculer un délai . . . 27
Arrondir à la semaine (pour les courbes épidémiques) . . . 27
Piège . . . 27
10. Import Excel et reformatage . . . 27
Import Excel . . . 27
Nettoyer les noms de colonnes . . . 27
Reformater : pivot_longer et pivot_wider . . . 28
Exercice concret : données ouvertes Géodes . . . 28
11. Écrire ses propres fonctions . . . 29
Syntaxe de base . . . 29
Exemple SP : IC 95% d'une proportion . . . 29
Pourquoi c'est essentiel . . . 30
12. Manipulation de texte — stringr . . . 30
Fonctions essentielles . . . 30
Exemple concret : nettoyer des communes . . . 31
Mini-guide des expressions régulières (regex) . . . 31
Utilisation dans filter() . . . 31
13. Itérer — appliquer la même opération à plusieurs éléments . . . 31
across() — appliquer une fonction à plusieurs colonnes . . . 31
map() — appliquer une fonction à une liste (purrr) . . . 32
Exemple SP : analyser 13 régions (conceptuel — nécessite un data frame avec colonnes `region` et `cas`) . . . 32
14. Exporter ses résultats . . . 32
Exporter un tableau vers Word . . . 32
Exporter vers Excel . . . 33
Sauvegarder un graphique . . . 33

PARTIE II — STATISTIQUES ET ANALYSES **33**

15. gtsummary — Tableau 1 . . . 33
    - Installation . . . 34
    - Tableau descriptif simple . . . 34
    - Tableau 1 comparatif (par groupe) + p-values . . . 34
    - Personnaliser le format . . . 34
    - Forcer le type d'une variable . . . 35
    - Tests choisis automatiquement par add_p() . . . 35
    - Alternative rapide : `janitor::tabyl()` . . . 35
    - Comment décrire un Tableau 1 dans la thèse . . . 35
16. Tests statistiques — Théorie et pratique . . . 36
    - Principe du test statistique . . . 36
    - Le seuil alpha = 0.05 . . . 36
    - Quel test choisir ? . . . 36
    - Syntaxe en R . . . 36
    - Lire les résultats . . . 37
    - Tests post-hoc — quels groupes diffèrent ? . . . 37
    - Vérifier la normalité avant un test paramétrique . . . 37
    - Comparaisons multiples . . . 38
    - OR vs Risque Relatif (RR) . . . 38
17. Régression logistique . . . 39
    - Vocabulaire . . . 39
    - Pourquoi "logistique" ? . . . 39
    - Pourquoi "binomial" ? . . . 39
    - Lire un OR . . . 39
    - Univariable vs Multivariable . . . 39
    - Étape 1 — Créer la variable binaire Y . . . 40
    - Étape 2 — Lancer la régression . . . 40
    - Lire le summary() . . . 40
    - Étape 3 — Tableau publication-ready avec finalfit . . . 40
    - Étape 4 — Forest plot (visualiser les OR) . . . 41
    - Les maths derrière (résumé) . . . 41
    - Vérifier le modèle logistique . . . 41

PARTIE III — ANALYSES PREDICT-R (Tutoriels complets) **42**

18. Données simulées Predict-R . . . 42
19. CJP — Proportion rattrapée . . . 43
    - Contexte . . . 43
    - Le concept mathématique . . . 43
    - Étape 1 — Identifier les patients rattrapés . . . 43
    - Étape 2 — Calculer la proportion et l'IC Wilson . . . 44
    - Étape 3 — Test binomial unilatéral . . . 44
20. Kappa de Cohen — Concordance . . . 45
    - Contexte . . . 45
    - Le concept mathématique . . . 45
    - Échelle d'interprétation (Landis & Koch, 1977) . . . 45
    - Le code . . . 45
    - Kappa pondéré — pour variables ordinales . . . 46
21. Se, Sp, VPP, VPN — Performance diagnostique . . . 46
    - Contexte . . . 46
    - Le cours : le tableau 2x2 . . . 46
    - Le code . . . 47
22. Taux de participation et complétion . . . 47
    - Contexte . . . 47
    - Le code . . . 48
23. Satisfaction — Échelle de Likert . . . 48
    - Contexte . . . 48
    - Le code . . . 48
    - Interprétation . . . 49
24. Analyses en sous-groupes . . . 49
    - Contexte . . . 49
    - Préparer les tranches d'âge . . . 49
    - Sous-groupe 1 : MRC connue vs inconnue . . . 50
    - Sous-groupe 2 : par tranche d'âge . . . 50
    - Interprétation d'un tableau par sous-groupes . . . 50
25. Chi2 de tendance — Cochran-Armitage . . . 51
    - Contexte . . . 51
    - Chi2 classique vs Chi2 de tendance . . . 51
    - Le code . . . 51
    - Analyse de sensibilité — vérifier la robustesse des résultats . . . 52

PARTIE IV — OUTILS COMPLÉMENTAIRES (Aperçu Phase 3) **52**

26. Git — Contrôle de version . . . 52
    - Vocabulaire . . . 52
    - Premiers pas (dans le terminal RStudio) . . . 53
    - .gitignore pour un projet R . . . 53
    - Ressource : Learn Git Branching — tutoriel interactif gratuit . . . 53
27. SQL — Interroger des bases de données . . . 54
    - Équivalences dplyr <-> SQL . . . 54
    - Exemple . . . 54
    - Utiliser SQL depuis R . . . 54
    - Ressource : SQLBolt — tutoriel interactif gratuit (12 leçons, ~2h) . . . 54
28. Shiny — Applications web interactives . . . 55
    - Structure d'une app Shiny . . . 55
    - Publier en ligne . . . 55
    - Ressource : Shiny — Get Started — tutoriel officiel . . . 56
29. Cartographie — Visualiser des données géographiques . . . 56
    - Premiers pas avec sf + ggplot2 . . . 56
30. Analyse de survie — Kaplan-Meier et Cox . . . 56
    - Kaplan-Meier — courbe de survie . . . 56
    - Cox — régression sur le temps de survie . . . 57
31. Reproductibilité avancée — renv . . . 57

ANNEXES **57**

A. Bonnes pratiques . . . 58
    - Workflow de nettoyage de données (dans cet ordre) . . . 58
    - Principes généraux . . . 58

B. Pièges courants . . . 58
    - Messages d'erreur fréquents et solutions . . . 59

C. Raccourcis clavier AZERTY . . . 60

D. Encodage UTF-8 . . . 60

E. Lien avec Predict-R — Vue d'ensemble . . . 60
    - Packages utilisés — Résumé . . . 61

F. Glossaire épidémiologique . . . 62
    - Mesures de fréquence . . . 62
    - Mesures d'association . . . 62

Types d'études (du plus faible au plus fort niveau de preuve) 62
Les 3 grands biais 63
Facteur de confusion 63
Critères de causalité (Bradford Hill, 1965) 63
Standardisation des taux 63
Courbe épidémique 64
Diagramme de flux (flowchart) 64
Grilles de lecture d'articles 65
Vocabulaire des bases de données en santé (culture SP) 65
Veille et alerte sanitaire (vocabulaire ARS) 65
G. Bases mathématiques 66
La loi normale (Gauss) 66
Variance et écart-type 66
Le théorème central limite 66
Degrés de liberté (df) 66
Intervalle de confiance — construction 67
Puissance statistique 67

Livret R — Manuel Personnel

Alexis Texier — Formation santé publique 2026 Se remplit au fur et à mesure de ma formation. Chaque section = quelque chose que je sais faire.

Sommaire

- **PARTIE I — FONDAMENTAUX R**
  - 0. Installation et premiers concepts
  - 1. Les bases
  - 2. Data frames
  - 3. dplyr — Manipuler les données
  - 4. Données manquantes (NA)
  - 5. Fichiers CSV et répertoire de travail
  - 6. ggplot2 — Graphiques
  - 7. R Markdown — Rapports reproductibles
  - 8. Jointures de tables
  - 9. Dates avec lubridate
  - 10. Import Excel et reformatage
  - 11. Écrire ses propres fonctions

  - 12. Manipulation de texte — stringr
  - 13. Itérer — across et map
  - 14. Exporter ses résultats

- **PARTIE II — STATISTIQUES ET ANALYSES**
  - 15. gtsummary — Tableau 1
  - 16. Tests statistiques — Théorie et pratique
  - 17. Régression logistique

- **PARTIE III — ANALYSES PREDICT-R (Tutoriels complets)**
  - 18. Données simulées Predict-R
  - 19. CJP — Proportion rattrapée
  - 20. Kappa de Cohen — Concordance
  - 21. Se, Sp, VPP, VPN — Performance diagnostique
  - 22. Taux de participation et complétion
  - 23. Satisfaction — Échelle de Likert
  - 24. Analyses en sous-groupes
  - 25. Chi2 de tendance — Cochran-Armitage

- **PARTIE IV — OUTILS COMPLÉMENTAIRES (Aperçu Phase 3)**
  - 26. Git — Contrôle de version
  - 27. SQL — Interroger des bases de données
  - 28. Shiny — Applications web interactives
  - 29. Cartographie
  - 30. Analyse de survie — Kaplan-Meier et Cox
  - 31. Reproductibilité avancée — renv

- **ANNEXES**
  - A. Bonnes pratiques
  - B. Pièges courants
  - C. Raccourcis clavier AZERTY
  - D. Encodage UTF-8
  - E. Lien avec Predict-R — Vue d'ensemble
  - F. Glossaire épidémiologique
  - G. Bases mathématiques

PARTIE I — FONDAMENTAUX R

---

0. Installation et premiers concepts

Installer R et RStudio

1. Installer **R** d'abord : https://cloud.r-project.org/ (choisir Windows -> base -> télécharger)
2. Installer **RStudio** ensuite : https://posit.co/download/rstudio-desktop/ (choisir la version gratuite)
3. Ouvrir **RStudio** (pas R directement) — RStudio utilise R en arrière-plan

L'interface RStudio (4 panneaux)

- **En haut à gauche** : l'éditeur de scripts (`.R` ou `.Rmd`) — c'est là que tu écris ton code
- **En bas à gauche** : la console — R attend tes commandes (le symbole `>` signifie "prêt")
- **En haut à droite** : l'environnement — tu vois tes variables, leurs types et valeurs
- **En bas à droite** : fichiers, graphiques, aide, packages

Créer un script

`File -> New File -> R Script` -> un fichier `.R` apparaît en haut à gauche. - Sauvegarder : **Ctrl + S** - Un asterisque `*` dans l'onglet = modifications non sauvegardées

Exécuter du code

- **Dans un script (.R)** : place ton curseur sur la ligne et appuie sur **Ctrl + Entrée**. La ligne est envoyée à la console et exécutée.
- **Plusieurs lignes** : sélectionne les lignes puis **Ctrl + Entrée** — elles s'exécutent dans l'ordre.
- **Dans la console** : tape le code directement après le `>` et appuie sur **Entrée**. Utile pour tester, mais rien n'est sauvegardé.
- **Tout le script** : **Ctrl + Shift + S** (source) ou **Ctrl + Alt + R** (run all).

Concepts fondamentaux

Fonction : une fonction prend des données en entrée (entre parenthèses) et renvoie un résultat.

```r
mean(c(10, 20, 30))    # mean = la fonction, c(10,20,30) = l'entrée, 20 = le résultat
```

Les parenthèses sont obligatoires. Sans elles, R affiche le code source de la fonction au lieu de l'exécuter.

Commentaire : tout ce qui suit un `#` est ignoré par R. C'est pour toi, pour expliquer ton code.

```r
age <- 29    # ceci est un commentaire, R l'ignore
```

Package : une extension qui ajoute des fonctions à R. Comme une application sur ton téléphone.

```r
install.packages("tidyverse")    # installer (une seule fois, comme télécharger l'app)
library(tidyverse)                # charger (à chaque session, comme ouvrir l'app)
```

Le tidyverse est une collection de packages : dplyr (manipulation), ggplot2 (graphiques), readr (CSV), stringr (texte), tidyr (reformatage), lubridate (dates), forcats (facteurs).

Aide : pour voir la documentation d'une fonction :

```r
?mean          # ouvre la page d'aide de mean
help(mean)     # pareil
```

L'onglet Help (en bas à droite) affiche la documentation.

Erreurs et avertissements : - `Error` (en rouge) = R s'arrête, quelque chose ne va pas. Lis le message : il dit souvent ce qui manque. - `Warning` (en orange) = R continue mais te prévient d'un problème potentiel. Pas bloquant mais à vérifier. - Erreurs fréquentes : objet introuvable (faute de frappe), fonction introuvable (package pas chargé), parenthèse manquante.

---

1. Les bases

Console vs Script

- **Console** (en bas) : pour tester rapidement. Rien n'est sauvegardé.
- **Script** (en haut) : fichier `.R` que tu écris et sauvegardes. Reproductible.
- Exécuter une ligne du script : **Ctrl + Entrée**
- Toujours travailler dans un script, pas dans la console.

Types de données

Type	Exemple	Description
Nombre	`42`, `3.14`	Numeric
Texte	`"Bonjour"`	Character — toujours entre guillemets
Logique	`TRUE`, `FALSE`	Logical — résultat d'une comparaison

Variables

```
age <- 29           # assigner une valeur
nom <- "Alexis"     # texte entre guillemets
```

- `<-` pour assigner (raccourci : **Alt + -** dans RStudio)
- R est **sensible à la casse** : `dfg` != `DFG` != `Dfg`

Vecteurs

Un vecteur = une liste de valeurs du même type.

```
ages <- c(52, 67, 43, 71, 58)    # c() = combine
noms <- c("Ali", "Bea", "Cam")
```

Fonctions de base

```
mean(ages)       # moyenne
median(ages)     # médiane
length(ages)     # combien d'éléments
summary(ages)    # résumé complet (min, max, quartiles, moyenne)
sum(ages)        # somme
```

Opérations logiques sur vecteurs

```r
ages > 60              # retourne TRUE/FALSE pour chaque élément
sum(ages > 60)         # compte combien sont > 60
mean(ages > 60)        # proportion (entre 0 et 1) qui sont > 60
```

> **Note statistique** : `mean()` sur un vecteur logique (TRUE/FALSE) donne une **proportion**. R traite TRUE = 1 et FALSE = 0. C'est la base du calcul du CJP de Predict-R.

Nettoyer l'environnement

```r
rm(list = ls())    # supprime toutes les variables --- utile quand on débute
```

> **Bonne pratique professionnelle** : `rm(list = ls())` est une béquille de débutant. Elle ne détache pas les packages chargés et ne remet pas les options par défaut. La vraie méthode : **redémarrer R** avec **Ctrl + Shift + F10** puis ré-exécuter le script. Tu comprendras mieux quand on verra les projets RStudio (section 5b).

---

2. Data frames

Un data frame = un tableau de données (lignes = observations, colonnes = variables).

```r
# Ce data frame est utilisé dans les sections 3 à 6 --- exécuter ce bloc en premier
patients <- data.frame(
  patient_id = 1:6,
  age = c(52, 67, 43, 71, 58, 49),
  sexe = c("F", "M", "F", "M", "F", "M"),
  dfg = c(85, 42, 95, 38, 72, 55),
  risque_pr = c("vert", "rouge", "vert", "rouge", "orange", "orange")
)
```

Ce data frame contient les colonnes utilisées dans toutes les sections suivantes : `age`, `sexe`, `dfg` (débit de filtration glomérulaire), `risque_pr` (niveau de risque Predict-R).

Accéder à une colonne

`$` permet d'extraire une colonne d'un data frame. C'est comme dire "donne-moi cette colonne".

```r
patients$age           # $ extrait une colonne -> donne le vecteur des âges
mean(patients$age)     # moyenne d'âge
```

Explorer un data frame

```r
glimpse(patients)      # aperçu compact --- nécessite library(tidyverse)
head(patients)         # 6 premières lignes
nrow(patients)         # nombre de lignes
ncol(patients)         # nombre de colonnes
```

Factor — variables catégorielles

Un **factor** est la façon dont R représente les variables catégorielles. R connaît les valeurs possibles (les "levels") et leur ordre.

```r
# Convertir en factor
sexe_f <- factor(c("F", "M", "F", "M"))
levels(sexe_f)    # [1] "F" "M" --- les catégories connues

# Factor ordonné (important pour les graphiques et tests)
risque <- factor(c("vert", "orange", "rouge"),
                 levels = c("vert", "orange", "rouge"))    # force l'ordre
```

- `factor()` : convertit un vecteur en factor
- `levels = c(...)` : définit l'ordre des catégories (sinon R trie par alphabet)
- Beaucoup de fonctions statistiques (finalfit, gtsummary, ggplot2) ont besoin de factors

---

3. dplyr — Manipuler les données

dplyr fait partie du **tidyverse** (collection de packages pour la data science en R).

```r
library(tidyverse)    # charger tidyverse (inclut dplyr, ggplot2, etc.)
```

Les 5 verbes essentiels

Verbe	Ce qu'il fait	Exemple
`filter()`	Garder certaines **lignes**	`filter(patients, dfg < 60)`
`select()`	Garder certaines **colonnes**	`select(patients, patient_id, age)`
`arrange()`	**Trier** les lignes	`arrange(patients, dfg)`
`mutate()`	**Créer** une nouvelle colonne	`mutate(patients, mrc = dfg < 60)`
`summarise()`	**Résumer** en une statistique	`summarise(patients, moy = mean(dfg))`

Le pipe `|>`

Enchaîner les opérations de gauche à droite, comme une phrase :

```r
patients |>
  filter(dfg < 60) |>
  select(patient_id, age, dfg)
```

Se lit : "Prends patients, PUIS filtre ceux avec DFG < 60, PUIS garde 3 colonnes."

Équivalent sans pipe (pour comprendre) :

```
# Avec pipe (lisible)
patients |> filter(age > 60)

# Sans pipe (identique mais moins lisible)
filter(patients, age > 60)
```

Le pipe envoie ce qui est à gauche comme **premier argument** de la fonction à droite.

group_by() + summarise()

Calculer des statistiques **par groupe** :

```
patients |>
  group_by(risque_pr) |>
  summarise(
    n = n(),                    # n() = compte les lignes dans chaque groupe (fonction spéciale d
    dfg_moyen = mean(dfg),      # moyenne DFG par groupe
    age_moyen = mean(age)       # moyenne âge par groupe
  )
```

rename() — renommer des colonnes

> **Note** : les exemples suivants utilisent un data frame `esperance` qui vient d'un CSV d'espérance de vie par département (exercice Phase 1). Si tu ne l'as pas, remplace par `patients` ou charge-le : `esperance <- read_csv("esperance_vie.csv")`

```
esperance <- esperance |>
  rename(departement = 1, esp_vie = 2)   # par position (le plus simple)
```

- Syntaxe : `nouveau_nom = ancien_nom` ou `nouveau_nom = numéro_colonne`
- **Par position** (`= 1`, `= 2`) : le plus fiable quand les noms ont des accents, espaces, apostrophes
- `names(donnees)` -> voir les noms exacts des colonnes

separate() — séparer une colonne en deux

```
esperance <- esperance |>
  separate(departement, into = c("region", "departement"), sep = " - ")
```

- `into` = noms des nouvelles colonnes
- `sep` = le séparateur (être précis avec les espaces)
- Après `separate()`, penser à `str_trim()` pour enlever les espaces en trop :

```
esperance <- esperance |>
  mutate(region = str_trim(region), departement = str_trim(departement))
```

drop_na() / replace_na() — gérer les NA

```
donnees |> drop_na()           # supprime toutes les lignes avec au moins un NA
donnees |> drop_na(age)        # supprime seulement les lignes où age est NA
donnees |> replace_na(list(age = 0, dfg = 999))   # remplace les NA par des valeurs
```

- **Attention** : `replace_na` avec des valeurs numériques est dangereux pour une thèse
- Préférer `drop_na()` ou rapporter les NA sans les remplacer

slice_min() / slice_max() — trouver les extrêmes

```
esperance |> slice_max(esp_vie, n = 1)   # département avec la plus haute espérance de vie
esperance |> slice_min(esp_vie, n = 1)   # département avec la plus basse
esperance |> slice_max(esp_vie, n = 5)   # top 5
```

Enchaîner summarise + slice — top/flop par groupe

```
# Top 5 régions par espérance de vie moyenne
esperance |>
  group_by(region) |>
  summarise(esp_moyenne = mean(esp)) |>
  slice_max(esp_moyenne, n = 5)
```

- `slice_max` / `slice_min` va **après** `summarise()` avec `|>` — pas dedans !
- Utiliser le **nom de la colonne créée** dans summarise (ici `esp_moyenne`, pas `esp`)

arrange(desc()) — tri décroissant

```
esperance |> arrange(desc(esp_vie))     # du plus grand au plus petit
esperance |> arrange(esp_vie)           # du plus petit au plus grand (par défaut)
```

count() — tableaux de fréquence

```
patients |> count(risque_pr)              # effectif par groupe
patients |> count(risque_pr, sexe)        # tableau croisé
```

Raccourci pour `group_by() |> summarise(n = n())`.

distinct() — supprimer les doublons

```
patients |> distinct()                    # lignes entièrement dupliquées
patients |> distinct(patient_id, .keep_all = TRUE)   # doublons sur patient_id seulement
```

- `.keep_all = TRUE` : garde toutes les colonnes (pas seulement celle testée)
- En ARS, les fichiers contiennent souvent des doublons — toujours vérifier avec `nrow()` avant et après

Détecter les valeurs aberrantes (outliers)

```
# Méthode IQR (la même que les moustaches du boxplot)
Q1 <- quantile(patients$age, 0.25)
Q3 <- quantile(patients$age, 0.75)
IQR <- Q3 - Q1
patients |> filter(age < Q1 - 1.5 * IQR | age > Q3 + 1.5 * IQR)

# Visuel : le boxplot les montre automatiquement comme des points isolés
ggplot(patients, aes(y = age)) + geom_boxplot()
```

> Ne **jamais** supprimer un outlier sans raison clinique. Un âge de 105 ans est rare mais possible. Un âge de -3 est une erreur de saisie.

case_when() — créer des catégories

```
patients <- patients |>
  mutate(
    stade_mrc = case_when(
      dfg >= 90 ~ "G1 - Normal",
      dfg >= 60 ~ "G2 - Legere",
      dfg >= 30 ~ "G3 - Moderee",
      dfg < 30  ~ "G4-G5 - Severe"
    )
  )
```

- `~` = "alors" (sépare condition et résultat)
- Les conditions sont testées dans l'ordre : la première vraie gagne
- `TRUE ~ "valeur"` à la fin = le "sinon" — attrape tout ce qui reste
- **Stocker le résultat** : `patients <- patients |> mutate(...)` sinon la colonne n'est pas sauvegardée

ifelse() — version rapide pour 2 options

```
patients <- patients |>
  mutate(risque_eleve = ifelse(risque_pr == "rouge", 1, 0))
```

- `ifelse(condition, valeur_si_vrai, valeur_si_faux)`
- Plus court que `case_when` quand il n'y a que 2 résultats possibles

---

4. Données manquantes (NA)

`NA` = Not Available = valeur manquante. Très fréquent dans les vrais datasets.

Le problème

```r
mean(patients$dfg)        # retourne NA si une seule valeur est manquante !
```

La solution : na.rm = TRUE

```r
mean(patients$dfg, na.rm = TRUE)    # "NA remove = oui" -> ignore les NA
```

Compter les NA

```r
sum(is.na(patients$dfg))       # combien de NA dans la colonne dfg
summary(patients$dfg)           # montre les NA automatiquement
```

NA dans group_by + summarise

```r
patients |>
  group_by(risque_pr) |>
  summarise(
    n = n(),
    dfg_moyen = mean(dfg, na.rm = TRUE),
    na_count = sum(is.na(dfg))
  )
```

Dans une thèse, toujours rapporter le nombre de données manquantes par variable et par groupe.

summary() vs summarise()

	`summary()`	`summarise()`
Vient de	R de base	dplyr/tidyverse
Usage	Exploration rapide	Analyses structurées
Résultat	Texte	Data frame (tableau)
Personnalisable	Non	Oui — tu choisis les stats
Avec `group_by()`	Non	Oui

---

5. Fichiers CSV et répertoire de travail

CSV (Comma-Separated Values)

Format universel pour échanger des données.

```r
# Sauvegarder
write_csv(patients, "patients.csv")    # tidyverse (pas de row.names à gérer)

# Charger
donnees <- read_csv("nom_du_fichier.csv")    # tidyverse
glimpse(donnees)                              # vérifier le contenu
```

Répertoire de travail

R cherche et sauvegarde les fichiers dans un dossier par défaut.

```r
getwd()                 # "get working directory" --- où R travaille
setwd("C:/mon/dossier") # "set working directory" --- changer de dossier (débutant)
list.files()            # voir les fichiers dans le dossier actuel
```

Pièges : - Toujours utiliser `/` (pas \) dans les chemins : `"C:/Users/Alexis/..."` - Toujours mettre le chemin entre guillemets - R Markdown cherche dans le dossier du fichier `.Rmd` — le CSV doit être au même endroit - Si `read_csv("fichier.csv")` échoue -> vérifier avec `getwd()` et `list.files()`

> **Important** : `setwd()` rend tes scripts non reproductibles (le chemin est différent sur un autre ordinateur). C'est OK pour apprendre, mais en milieu professionnel, on utilise des **projets RStudio** (voir ci-dessous).

Projets RStudio (.Rproj) — la bonne pratique

Un **projet RStudio** = un dossier dédié avec un fichier `.Rproj`. Quand tu ouvres le projet, R se place automatiquement dans le bon dossier. Plus besoin de `setwd()`.

Créer un projet : `File -> New Project... -> New Directory -> New Project -> choisir nom et emplacement`

Structure recommandée :

```
mon-projet/
  mon-projet.Rproj       # fichier projet (double-cliquer pour ouvrir)
  data/                   # données brutes (CSV, XLSX)
  scripts/                # scripts R (.R)
  output/                 # résultats (tableaux, figures, rapports)
```

Utiliser `here::here()` pour les chemins :

```r
# install.packages("here")
library(here)

# Au lieu de :
donnees <- read_csv("C:/Users/Alexis/Documents/projet/data/patients.csv")

# Écrire :
donnees <- read_csv(here("data", "patients.csv"))
```

- `here("data", "patients.csv")` construit le chemin automatiquement à partir de la racine du projet
- Fonctionne sur n'importe quel ordinateur (Windows, Mac, Linux)
- **C'est le standard professionnel** — la première chose qu'on vérifie chez un candidat SpFrance

> **Pour ta thèse** : créer un projet `Predict-R/` avec `data/`, `scripts/`, `output/`. Mettre le CSV dans `data/` et tes scripts d'analyse dans `scripts/`.

---

6. ggplot2 — Graphiques

Grammaire : `ggplot(données, aes(...)) + geom_type() + options`

Structure de base

`aes()` = **aesthetics** (esthétique). C'est ici que tu dis à ggplot quelles **colonnes** utiliser pour les axes, les couleurs, etc. Tout ce qui **dépend des données** va dans `aes()`. Ce qui est **fixe** (une couleur unique) va directement dans le `geom_*()`.

```
ggplot(patients, aes(x = ..., y = ...)) +   # canevas + axes
  geom_...() +                               # type de graphique
  labs(title = "...", x = "...", y = "...") + # titres
  theme_minimal()                            # thème propre
```

Les graphiques de base

Histogramme — distribution d'une variable continue

```
# Avec le data frame patients (section 2)
ggplot(patients, aes(x = age)) +
  geom_histogram(bins = 5, fill = "steelblue", color = "white") +
  labs(title = "Distribution de l'age",
       x = "Age (annees)", y = "Nombre de patients") +
  theme_minimal()

# Avec un data frame externe (ex: espérance de vie par département)
# ggplot(esperance, aes(x = esp_vie)) +
#   geom_histogram(bins = 10, fill = "steelblue", color = "white")
```

- `bins` = nombre de barres (tester 5, 10, 15 pour voir)
- `fill` = couleur de remplissage, `color` = couleur du contour

Barplot — effectifs d'une variable catégorielle

```
ggplot(patients, aes(x = risque_pr, fill = risque_pr)) +
  geom_bar() +
  scale_fill_manual(values = c("vert" = "green3", "orange" = "orange", "rouge" = "red3"))
```

Boxplot — comparer une variable continue entre groupes

```
ggplot(patients, aes(x = sexe, y = dfg, fill = sexe)) +
  geom_boxplot(outlier.shape = NA) +
  labs(x = "Sexe", y = "DFG", fill = "Sexe")
```

- Lecture : trait central = médiane, boîte = Q1-Q3 (50% des données), moustaches = 1.5xIQR

  Note statistique : L'**IQR** (Interquartile Range) = Q3 - Q1. Les moustaches vont jusqu'à 1.5xIQR au-dessus de Q3 et en-dessous de Q1. Au-delà = outliers (points isolés).

Scatter plot — relation entre 2 variables continues

```
ggplot(patients, aes(x = age, y = dfg, color = risque_pr)) +
  geom_point(size = 3) +
  scale_color_manual(values = c("vert" = "green3", "orange" = "orange", "rouge" = "red3"))
```

Graphiques avancés

Barplot proportionnel (empilé 100%)

```
ggplot(patients, aes(x = risque_pr, fill = sexe)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  labs(title = "Repartition H/F par niveau de risque",
       x = "Risque Predict-R", y = "Proportion", fill = "Sexe") +
  theme_minimal()
```

- `position = "fill"` : chaque barre = 100%, montre les **proportions** au lieu des effectifs
- `scales::percent` : affiche 0%, 25%, 50%… (la notation `package::fonction` utilise une fonction d'un package sans faire `library()`)
- Utile pour comparer des groupes de tailles différentes

Violin plot — distribution + boxplot combinés

```
ggplot(patients, aes(x = risque_pr, y = age, fill = risque_pr)) +
  geom_violin() +
  geom_boxplot(width = 0.2, fill = "white") +
  scale_fill_manual(values = c("vert" = "green3",
                               "orange" = "orange",
                               "rouge" = "red2")) +
  labs(title = "Distribution de l'age par niveau de risque",
       x = "Risque Predict-R", y = "Age") +
  theme_classic() +
  theme(legend.position = "none")
```

- `geom_violin()` : forme = distribution (plus c'est large, plus il y a de patients à cette valeur)
- `geom_boxplot(width = 0.2, fill = "white")` : mini boxplot superposé à l'intérieur
- `width` = largeur (0.2 = petit boxplot à l'intérieur du violin)
- `theme(legend.position = "none")` : masque la légende quand l'axe X dit déjà la même chose
- Lecture : largeur = densité, boxplot blanc = médiane + quartiles

Density plot — histogramme lissé, superposer des groupes

```
ggplot(patients, aes(x = age, fill = risque_pr)) +
  geom_density(alpha = 0.5) +
  scale_fill_manual(values = c("vert" = "green3",
                               "orange" = "orange",
                               "rouge" = "red2")) +
```

```
  labs(title = "Distribution de l'age par niveau de risque",
       x = "Age", y = "Densite", fill = "Risque") +
  theme_minimal()
```

- `geom_density()` : courbe lissée (pas des barres)
- `alpha = 0.5` : transparence (0 = invisible, 1 = opaque) — indispensable pour voir les courbes superposées
- L'axe Y (densité) montre des petites valeurs (0.01, 0.02...) — c'est normal, on lit la **forme**, pas les valeurs Y

> **Note statistique** : La densité est une version normalisée de l'histogramme. L'aire sous la courbe = 1. Ça permet de comparer des groupes de tailles différentes sur le même graphique.

Couleur fixe vs couleur par variable

Situation	Où mettre la couleur	Exemple
Même couleur pour tout	Dans le `geom_*()`	`geom_histogram(fill = "steelblue")`
Couleur selon les données	Dans `aes()` + `scale_*_manual()`	`aes(fill = risque)` + `scale_fill_manual(...)`

- `fill` = remplissage (barres, boîtes) -> `scale_fill_manual()`
- `color` = contour/points -> `scale_color_manual()`

Facettes — sous-panneaux

facet_wrap() — une variable

```
ggplot(patients, aes(x = age, fill = sexe)) +
  geom_histogram(bins = 10, color = "white") +
  facet_wrap(~ risque_pr) +
  theme_minimal()
```

- `facet_wrap(~ variable)` : un panneau par valeur de la variable
- `~` se lit "en fonction de"

facet_grid() — deux variables (grille)

```
ggplot(patients, aes(x = age)) +
  geom_histogram(bins = 8, fill = "steelblue", color = "white") +
  facet_grid(sexe ~ risque_pr) +
  theme_minimal()
```

- `facet_grid(lignes ~ colonnes)` : grille croisée

Thèmes — apparence du graphique

Thème	Style	Usage
`theme_minimal()`	Épuré, pas de cadre	Exploration
`theme_classic()`	Axes en L, fond blanc	**Publications scientifiques**
`theme_bw()`	Cadre complet, fond blanc	Propre, formel
`theme_light()`	Comme bw, grille légère	Présentations
`theme_void()`	Rien du tout	Pie charts, cartes

Personnaliser le texte

```r
theme_classic() +
theme(
  plot.title = element_text(size = 14, face = "bold"),
  axis.title = element_text(size = 12),
  axis.text = element_text(size = 10),
  legend.position = "none"    # masquer la légende si inutile
)
```

- `size` : taille de police
- `face` : `"bold"`, `"italic"`, `"bold.italic"`
- `legend.position` : `"none"` (masquer), `"right"` (défaut), `"bottom"`, `"top"`

Ordonner les barres par valeur (pas par alphabet)

```r
# Sans fct_reorder : barres triées par alphabet (Ariège, Haute-Garonne, Tarn...)
# Avec fct_reorder : barres triées par la valeur -> plus lisible
library(forcats)    # inclus dans tidyverse
ggplot(donnees, aes(x = fct_reorder(departement, taux), y = taux)) +
  geom_col(fill = "steelblue") +
  coord_flip() +                  # barres horizontales (lisible si beaucoup de catégories)
  labs(x = "", y = "Taux") +
  theme_minimal()
```

- `fct_reorder(variable, valeur)` : réordonne les catégories par la valeur (croissant)
- `coord_flip()` : pivote le graphique (barres horizontales — indispensable avec beaucoup de départements)

Quand utiliser quel graphique ?

Graphique	Quand l'utiliser
Histogramme	1 variable continue, vue détaillée des effectifs
Density	Comparer la distribution de 2-3 groupes superposés

Graphique	Quand l'utiliser
Boxplot	Compact, comparaison rapide médiane/quartiles entre groupes
Violin	Montrer la forme de la distribution + médiane/quartiles
Barplot	Effectifs d'une variable catégorielle
Barplot proportionnel	Comparer des proportions entre groupes de tailles différentes
Scatter	Relation entre 2 variables continues
Facettes	Éclater n'importe quel graphique par sous-groupe

---

7. R Markdown — Rapports reproductibles

Un fichier `.Rmd` = texte + code R -> génère un rapport HTML ou PDF automatiquement. **Knit** = "tricoter" — R exécute le code, insère les résultats dans le texte, et crée un fichier HTML.

Créer un R Markdown

`File -> New File -> R Markdown...` -> choisir titre, auteur, format (HTML).

En-tête basique

```
---
title: "Mon rapport"
author: "Alexis Texier"
date: "2026-03-12"
output: html_document
---
```

En-tête avancé (rapport de thèse)

```
---
title: "Rapport avance - Predict-R"
author: "Alexis Texier"
date: "2026-03-24"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    theme: cosmo
---
```

Option	Ce que ça fait
`toc: true`	Table des matières automatique
`toc_float: true`	Sommaire flottant sur le côté (suit le scroll)
`number_sections: true`	Numérotation 1, 1.1, 1.2...
`theme: cosmo`	Thème visuel (`cosmo`, `flatly`, `journal`, `readable`)

Blocs de code (chunks)

Un **chunk** = un bloc de code R dans le .Rmd, délimité par trois backticks. Raccourci pour en insérer un : **Ctrl + Alt + I**.

```
```{r}
mean(donnees$dfg, na.rm = TRUE)
```
```

Bloc setup — charger packages et données (invisible)

```
```{r setup, include=FALSE}
library(tidyverse)
library(gtsummary)
patients <- read_csv("predict_r_sim.csv")
knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE)
```
```

> C'est la ligne "magique" qui configure tous les chunks d'un coup. Copie-la telle quelle. `knitr` = le package qui "tricote", `opts_chunk` = les options des blocs, `$set()` = définir ces options.

Options des chunks

Option	Ce que ça fait
`include = FALSE`	Le bloc est **totalement invisible** (code + résultat) — pour le setup
`echo = FALSE`	Cache le **code** R, montre seulement le résultat (tableau, graphique)
`echo = TRUE`	Montre le code (utile pour un bloc spécifique dans un rapport sans code)
`message = FALSE`	Cache les **messages** d'info (ex: "Rows: 30 Columns: 5")
`warning = FALSE`	Cache les **avertissements** (ex: "removed 2 rows")
`fig.width = 8`	Largeur de la figure en pouces
`fig.height = 5`	Hauteur de la figure en pouces

Options globales vs locales

- **Globales** (dans le setup) : `knitr::opts_chunk$set(echo = FALSE)` -> s'applique à tous les chunks

- **Locales** (dans un chunk) : `{r, echo=TRUE}` -> s'applique seulement à ce chunk
- L'option locale a priorité sur l'option globale

> **Quarto** : depuis 2023, **Quarto** (`.qmd`) remplace progressivement R Markdown. La syntaxe est très similaire, mais Quarto fonctionne aussi avec Python et Julia. SpFrance et l'EHESP migrent vers Quarto. Pour l'instant, R Markdown suffit — quand tu seras à l'aise, la transition sera facile (`File -> New File -> Quarto Document`). Voir quarto.org.

Sections

```
# Titre principal       (niveau 1 --- apparaît dans la table des matières)
## Sous-titre            (niveau 2 --- 3.1, 3.2...)
### Sous-sous-titre      (niveau 3)
```

Exemple de rapport complet

```
‘‘‘{r setup, include=FALSE}
library(tidyverse)
library(gtsummary)
patients <- read_csv("C:/chemin/predict_r_sim.csv")
knitr::opts_chunk$set(echo = FALSE, message = FALSE, warning = FALSE)
‘‘‘

## Introduction
Ce rapport presente l’analyse des donnees simulees Predict-R.

## Resultats

### Tableau descriptif
‘‘‘{r}
patients |>
  select(age, sexe, dfg, risque_pr) |>
  tbl_summary(by = risque_pr) |>
  add_p() |>
  add_overall()
‘‘‘

### Distribution de l’age
‘‘‘{r, fig.width=8, fig.height=5}
ggplot(patients, aes(x = risque_pr, y = age, fill = risque_pr)) +
  geom_violin() +
  geom_boxplot(width = 0.2, fill = "white") +
  theme_classic()
‘‘‘

## Conclusion
Les patients a risque eleve sont plus ages.
```

Important

- Le `.Rmd` cherche les fichiers dans **son propre dossier** (pas le working directory de la console)
- Le CSV doit être au même endroit que le `.Rmd`, ou utiliser le chemin complet (`"C:/..."`)
- Si le nom du fichier .Rmd contient des accents -> préférer des noms sans accents

8. Jointures de tables

En santé publique, tu croiseras toujours plusieurs tables : patients + résultats de labo, cas de maladie + communes, etc. La jointure = fusionner deux tables qui partagent une colonne commune.

Les 3 jointures essentielles

Jointure	Ce qu'elle garde
`left_join(x, y)`	Toutes les lignes de x + colonnes de y quand ça correspond
`inner_join(x, y)`	Seulement les lignes présentes dans les DEUX tables
`full_join(x, y)`	Tout, des deux côtés (NA si pas de correspondance)

Exemple concret

```r
# Table 1 : patients
patients <- data.frame(
  patient_id = c(1, 2, 3, 4),
  age = c(52, 67, 43, 71),
  commune_code = c("31555", "31555", "09122", "31088")
)

# Table 2 : communes
communes <- data.frame(
  commune_code = c("31555", "31088", "09122"),
  nom_commune = c("Toulouse", "Colomiers", "Foix"),
  region = c("Occitanie", "Occitanie", "Occitanie")
)

# Jointure : ajouter le nom de commune à chaque patient
patients_complets <- patients |>
  left_join(communes, by = "commune_code")

patients_complets
#   patient_id age commune_code nom_commune    region
# 1          1  52        31555    Toulouse Occitanie
# 2          2  67        31555    Toulouse Occitanie
# 3          3  43        09122        Foix Occitanie
# 4          4  71        31088   Colomiers Occitanie
```

- `by = "commune_code"` : la colonne commune aux deux tables

- `left_join` : garde tous les patients, même si leur commune n'est pas dans la table communes (-> NA)
- Si une colonne a le même nom dans les deux tables (autre que `by`), R ajoute `.x` et `.y` pour les distinguer

  C'est l'équivalent du `LEFT JOIN` en SQL.

---

9. Dates avec lubridate

En surveillance épidémiologique, les dates sont partout : date de symptômes, date de déclaration, semaines épidémiologiques.

Charger lubridate

```
library(lubridate)    # inclus dans tidyverse
```

Convertir du texte en date

R ne comprend pas automatiquement que "25/03/2026" est une date. Il faut lui dire le format.

```
# Format français : jour/mois/année
dmy("25/03/2026")          # -> "2026-03-25"
dmy("25 mars 2026")        # -> fonctionne aussi

# Format international : année-mois-jour
ymd("2026-03-25")          # -> "2026-03-25"

# Format américain : mois/jour/année
mdy("03/25/2026")          # -> "2026-03-25"
```

Retiens : `dmy` = **d**ay-**m**onth-**y**ear, `ymd` = **y**ear-**m**onth-**d**ay.

Extraire des composants

```
date <- dmy("25/03/2026")
year(date)      # 2026
month(date)     # 3
day(date)       # 25
wday(date, label = TRUE)    # mar (mardi)
week(date)      # numéro de semaine

# Semaine ISO (standard officiel BEH / Géodes / SpFrance)
isoweek(date)   # 13 (semaine 13 de l'année)
isoyear(date)   # 2026 (année ISO --- peut différer au 31 décembre)
```

> En surveillance, on utilise les **semaines ISO** (S01 à S52/53), pas `week()`. C'est le format du BEH et de Géodes.

Calculer un délai

```
# Exemple : délai entre symptômes et déclaration
cas <- data.frame(
  date_symptomes = dmy(c("10/03/2026", "15/03/2026", "20/03/2026")),
  date_declaration = dmy(c("12/03/2026", "18/03/2026", "21/03/2026"))
)

cas <- cas |>
  mutate(delai_jours = as.numeric(date_declaration - date_symptomes))

cas$delai_jours    # 2, 3, 1
```

Arrondir à la semaine (pour les courbes épidémiques)

```
cas <- cas |>
  mutate(semaine = floor_date(date_symptomes, "week"))
```

`floor_date(date, "week")` arrondit la date au lundi de sa semaine. Utile pour regrouper les cas par semaine dans un graphique.

Piège

Le format français (jour/mois) est l'inverse du format américain (mois/jour). Si tes dates sont dans un CSV, vérifie toujours le format avant de convertir. `01/03/2026` = 1er mars en français, 3 janvier en américain.

---

10. Import Excel et reformatage

Import Excel

En ARS, la majorité des fichiers arrivent en `.xlsx`, pas en `.csv`.

```
# install.packages("readxl")
library(readxl)

donnees <- read_excel("fichier.xlsx")                  # première feuille
donnees <- read_excel("fichier.xlsx", sheet = 2)       # deuxième feuille
donnees <- read_excel("fichier.xlsx", sheet = "Données") # par nom de feuille
```

Nettoyer les noms de colonnes

Les fichiers Excel ont souvent des noms de colonnes avec des espaces, accents, parenthèses.

```r
# install.packages("janitor")
library(janitor)
donnees <- donnees |> clean_names()
# "Nb de Cas Confirmés (2024)" -> "nb_de_cas_confirmes_2024"
```

Reformater : pivot_longer et pivot_wider

Les données de surveillance arrivent souvent en format **large** (une colonne par semaine) :

commune	sem_01	sem_02	sem_03
Toulouse	5	8	3
Foix	1	2	0

Pour ggplot2, tu as besoin du format **long** (une ligne par observation) :

commune	semaine	cas
Toulouse	sem_01	5
Toulouse	sem_02	8
...	...	...

```r
library(tidyr)    # inclus dans tidyverse

# Large -> Long
donnees_long <- donnees |>
  pivot_longer(
    cols = starts_with("sem_"),     # colonnes à pivoter
    names_to = "semaine",           # nom de la nouvelle colonne "catégorie"
    values_to = "cas"               # nom de la nouvelle colonne "valeur"
  )

# Long -> Large (l'inverse)
donnees_large <- donnees_long |>
  pivot_wider(
    names_from = semaine,           # colonne qui deviendra les noms de colonnes
    values_from = cas               # colonne qui deviendra les valeurs
  )
```

Exercice concret : données ouvertes Géodes

Géodes (https://geodes.santepubliquefrance.fr/) = la plateforme de SpFrance avec 900+ indicateurs de santé. Voici comment l'utiliser :

1. Aller sur Géodes -> choisir un indicateur (ex : "Diabète traité" -> "Prévalence")
2. Sélectionner le niveau géographique (département) et la période
3. Cliquer "Télécharger" -> CSV

```r
library(tidyverse)
library(janitor)

# Charger les données Géodes
geodes <- read_csv("export_geodes.csv", skip = 3) |>    # skip = 3 : sauter les lignes d'en-tête
  clean_names()

glimpse(geodes)    # vérifier les noms de colonnes

# Visualiser : prévalence du diabète par département
ggplot(geodes, aes(x = fct_reorder(departement, prevalence), y = prevalence)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Prevalence du diabete par departement",
       x = "", y = "Prevalence (%)") +
  theme_minimal()
```

> **Note** : les fichiers Géodes ont souvent des en-têtes spéciaux (lignes de métadonnées avant les données). `skip = 3` saute ces lignes. Vérifier avec un éditeur de texte si le nombre est correct.

---

11. Écrire ses propres fonctions

Quand tu répètes le même code 3 fois -> écrire une **fonction**. C'est le saut entre "je sais utiliser R" et "je suis autonome en R".

Syntaxe de base

```r
# Définir une fonction
calcul_imc <- function(poids, taille_m) {
  imc <- poids / (taille_m ^ 2)
  return(imc)
}

# Utiliser la fonction
calcul_imc(75, 1.80)       # 23.1
calcul_imc(90, 1.65)       # 33.1
```

- `function(arguments)` : définit les entrées
- `{ ... }` : le code à exécuter
- `return(resultat)` : ce que la fonction renvoie (optionnel si c'est la dernière ligne)

Exemple SP : IC 95% d'une proportion

```r
# IC par approximation de Wald (simplifié pour apprendre function())
# Pour la thèse, utiliser prop.test() qui donne l'IC Wilson (plus fiable)
ic_proportion <- function(x, n, niveau = 0.95) {
```

```r
  p <- x / n
  z <- qnorm(1 - (1 - niveau) / 2)    # 1.96 pour 95%
  se <- sqrt(p * (1 - p) / n)
  ic_bas <- p - z * se
  ic_haut <- p + z * se
  cat("Proportion :", round(p * 100, 1), "%\n")
  cat("IC", niveau * 100, "% : [", round(ic_bas * 100, 1), "% ;",
      round(ic_haut * 100, 1), "%]\n")
}

ic_proportion(23, 150)    # 23 rattrapés sur 150
```

Pourquoi c'est essentiel

- En poste, tu analyseras 13 régions, 101 départements, 52 semaines -> mêmes calculs, données différentes
- Une fonction = un seul endroit à corriger si tu trouves une erreur
- Les packages R ne sont que des collections de fonctions

---

12. Manipulation de texte — stringr

En ARS, tu nettoieras des noms de communes, des codes CIM-10, des adresses. `stringr` (inclus dans tidyverse) est le package pour ça.

Fonctions essentielles

```r
library(tidyverse)    # stringr est inclus

# Détecter un motif
str_detect("Toulouse", "ouse")        # TRUE
str_detect("Foix", "ouse")            # FALSE

# Remplacer
str_replace("Dr. Martin", "Dr.", "Docteur")    # "Docteur Martin"

# Extraire (avec une expression régulière --- mini-guide regex ci-dessous)
str_extract("CIM-10: E11.9", "[A-Z]\\d{2}")    # "E11" (code CIM-10)

# Mettre en minuscules / majuscules
str_to_lower("TOULOUSE")    # "toulouse"
str_to_upper("toulouse")    # "TOULOUSE"

# Coller des chaînes
str_c("ARS", "Occitanie", sep = " ")    # "ARS Occitanie"
```

Exemple concret : nettoyer des communes

```r
communes <- data.frame(
  nom = c("  Toulouse ", "FOIX", "saint-gaudens", "Castres Cedex")
)

communes <- communes |>
  mutate(
    nom_propre = nom |>
      str_trim() |>                    # enlever espaces début/fin
      str_to_title() |>                # Première Lettre En Majuscule
      str_replace(" Cedex", "")        # enlever "Cedex"
  )
```

Mini-guide des expressions régulières (regex)

Les regex sont des motifs de recherche dans du texte. Tu n'as pas besoin de tout retenir — ce tableau suffit :

Regex	Signification	Exemple
`.`	N'importe quel caractère	`"a.c"` trouve "abc", "a1c"
`^`	Commence par	`"^E"` trouve "E11" mais pas "AE"
`$`	Finit par	`"9$"` trouve "E11.9"
`\\d`	Un chiffre	`"\\d{2}"` = exactement 2 chiffres
`[A-Z]`	Une lettre majuscule	`"[A-Z]\\d{2}"` = "E11"
`*`	0 ou plus	`"ab*c"` trouve "ac", "abc", "abbc"

En R, les `\` doivent être doublés : `\d` en regex s'écrit `"\\d"` en R.

Utilisation dans filter()

```r
# Garder seulement les lignes qui contiennent "Saint"
communes |> filter(str_detect(nom, "Saint"))

# Garder les codes CIM-10 commençant par E (maladies endocriniennes)
cim |> filter(str_detect(code, "^E"))    # ^ = "commence par"
```

---

13. Itérer — appliquer la même opération à plusieurs éléments

across() — appliquer une fonction à plusieurs colonnes

```r
# Arrondir toutes les colonnes numériques à 1 décimale
patients |>
  mutate(across(where(is.numeric), ~ round(.x, 1)))
```

```r
# Calculer la moyenne de plusieurs colonnes par groupe
patients |>
  group_by(risque_pr) |>
  summarise(across(c(age, dfg),
                   list(moy = ~ mean(.x, na.rm = TRUE),
                        sd = ~ sd(.x, na.rm = TRUE)),
                   .names = "{.col}_{.fn}"))
```

- `across(colonnes, fonction)` : applique la fonction à chaque colonne sélectionnée
- `where(is.numeric)` : toutes les colonnes numériques
- `~ round(.x, 1)` : **formule anonyme** — le `~` signifie "applique cette opération", `.x` est un placeholder pour "l'élément en cours". C'est comme dire "pour chaque colonne, arrondir à 1 décimale"
- `.names = "{.col}_{.fn}"` : nomme les résultats (ex: `age_moy`, `dfg_sd`)

map() — appliquer une fonction à une liste (purrr)

```r
library(purrr)    # inclus dans tidyverse

# Lire tous les CSV d'un dossier d'un coup
fichiers <- list.files("data/", pattern = "\\.csv$", full.names = TRUE)
donnees <- map(fichiers, read_csv)        # liste de data frames
donnees <- map_dfr(fichiers, read_csv)    # tout empilé en un seul data frame
```

- `map(liste, fonction)` : applique la fonction à chaque élément -> retourne une liste
- `map_dfr()` : pareil mais empile les résultats en un seul data frame (dfr = data frame rows)

Exemple SP : analyser 13 régions (conceptuel — nécessite un data frame avec colonnes `region` et `cas`)

```r
# Calculer l'incidence par région automatiquement
resultats <- patients |>
  group_by(region) |>
  group_split() |>                          # sépare en sous-tables par région
  map(~ tibble(
    region = unique(.x$region),
    n_cas = sum(.x$cas),
    incidence = sum(.x$cas) / nrow(.x) * 1000
  )) |>
  bind_rows()                               # recolle tout
```

---

14. Exporter ses résultats

Exporter un tableau vers Word

```r
# install.packages("flextable")
library(flextable)
```

```r
library(gtsummary)    # si pas déjà chargé (voir section 15)

tableau <- patients |>
  tbl_summary(by = risque_pr) |>
  add_p()

# Convertir en flextable puis sauvegarder
tableau |>
  as_flex_table() |>
  save_as_docx(path = "output/tableau1.docx")
```

Exporter vers Excel

```r
# install.packages("writexl")
library(writexl)

write_xlsx(patients, "output/patients.xlsx")

# Plusieurs feuilles
write_xlsx(list(
  "Patients" = patients,
  "Résultats" = resultats
), "output/analyse_complete.xlsx")
```

Sauvegarder un graphique

```r
# Sauvegarder le dernier graphique affiché
ggsave("output/figure1.png", width = 8, height = 5, dpi = 300)

# Sauvegarder un graphique spécifique
mon_plot <- ggplot(patients, aes(x = age)) + geom_histogram()
ggsave("output/histogramme.png", plot = mon_plot, width = 8, height = 5, dpi = 300)
```

- `dpi = 300` : qualité d'impression (300 = standard publication)
- Formats : `.png` (figures), `.pdf` (vectoriel, meilleur pour publications), `.svg`

PARTIE II — STATISTIQUES ET ANALYSES

---

15. gtsummary — Tableau 1

Package qui génère automatiquement les tableaux descriptifs des articles médicaux.

Installation

```r
install.packages("gtsummary")
library(gtsummary)
```

Tableau descriptif simple

```r
patients |>
  select(age, sexe, dfg, risque_pr) |>
  tbl_summary()
```

- Variables continues -> Médiane (Q1, Q3)
- Variables catégorielles -> n (%)

Tableau 1 comparatif (par groupe) + p-values

```r
patients |>
  select(age, sexe, dfg, risque_pr) |>
  tbl_summary(by = risque_pr) |>
  add_p() |>
  add_overall()
```

- `by = variable` -> une colonne par groupe
- `add_p()` -> p-values automatiques (choisit le bon test)
- `add_overall()` -> colonne "Total"

Personnaliser le format

```r
patients |>
  select(age, sexe, dfg, risque_pr) |>
  tbl_summary(
    by = risque_pr,
    statistic = list(
      all_continuous() ~ "{mean} ({sd})",
      all_categorical() ~ "{n} ({p}%)"
    ),
    label = list(
      age ~ "Age (annees)",
      sexe ~ "Sexe",
      dfg ~ "DFG (mL/min/1.73m2)"
    )
  ) |>
  add_p() |>
  add_overall()
```

- `statistic` -> choisir moyenne (SD) au lieu de médiane (IQR)
- `label` -> noms propres pour les variables
- `~` = "est décrit par" (sépare variable et format)

- `all_continuous()` = "toutes les variables continues" (sélecteur gtsummary — raccourci au lieu de lister chaque variable)
- `all_categorical()` = "toutes les variables catégorielles"
- `everything()` = "toutes les variables" (pratique avec `type = list(everything() ~ "continuous")`)
- Placeholders disponibles : `{mean}`, `{sd}`, `{median}`, `{p25}`, `{p75}`, `{n}`, `{p}`

Forcer le type d'une variable

```
tbl_summary(
  type = list(nb_fdr_has ~ "continuous"),   # traiter comme nombre, pas catégorie
  ...
)
```

- gtsummary devine le type : peu de valeurs distinctes (1-5) -> catégoriel, beaucoup -> continu
- `type = list(variable ~ "continuous")` -> forcer le traitement en continu (utile pour Likert, scores)

Tests choisis automatiquement par add_p()

Type de variable	2 groupes	3+ groupes
Continue	Wilcoxon	Kruskal-Wallis
Catégorielle (effectifs >= 5)	Chi2	Chi2
Catégorielle (effectifs < 5)	Fisher exact	Fisher exact

Alternative rapide : `janitor::tabyl()`

Pour un tableau croisé rapide (équivalent des tableaux croisés dynamiques Excel), `tabyl()` est plus simple que `tbl_summary()` :

```
library(janitor)
patients |> tabyl(risque_pr, sexe)                # tableau croisé
patients |> tabyl(risque_pr, sexe) |> adorn_percentages("row") |> adorn_ns()
# -> pourcentages en ligne avec effectifs entre parenthèses
```

Très utilisé en ARS pour des explorations rapides. Pour un article/thèse, reste sur `gtsummary`.

Comment décrire un Tableau 1 dans la thèse

1. Dire ce que le tableau montre (variables, stratification)
2. Signaler les différences **significatives** ($p < 0.05$) avec le sens de la différence
3. Mentionner brièvement que les autres variables sont **comparables**
4. Ne jamais dire "significatif" sans donner la p-value

   Exemple : "La répartition par sexe différait significativement selon la tranche d'âge ($p = 0.003$), avec une proportion de femmes plus élevée chez les moins de 50 ans (73%) que chez les 65-74 ans (51%). Les autres caractéristiques étaient comparables entre les groupes."

---

16. Tests statistiques — Théorie et pratique

Principe du test statistique

Un test statistique répond toujours à la même question : **"Est-ce que la différence que j'observe est due au hasard, ou est-elle réelle ?"**

Le raisonnement : 1. On pose une **hypothèse nulle H0** : "il n'y a pas de différence" (ou "pas d'effet") 2. On pose une **hypothèse alternative H1** : "il y a une différence" 3. Le test calcule une **p-value** : la probabilité d'observer cette différence (ou plus extrême) si H0 était vraie 4. Si p < 0.05 -> la probabilité est trop faible -> on **rejette H0** -> différence significative 5. Si p >= 0.05 -> on **ne rejette pas H0** -> pas de preuve suffisante

> **Attention** : p >= 0.05 ne veut PAS dire "il n'y a pas de différence". Ça veut dire "on n'a pas assez de preuves pour conclure". La nuance est importante dans une thèse.

Le seuil alpha = 0.05

- C'est une **convention**, pas une loi de la nature
- alpha = 0.05 signifie : on accepte un risque de 5% de se tromper (conclure qu'il y a une différence alors qu'il n'y en a pas = erreur de type I)
- En cancérologie, on utilise parfois alpha = 0.01 (plus strict)
- Ne JAMAIS modifier alpha après avoir vu les résultats -> c'est du **p-hacking**

Quel test choisir ?

Question	Variables	Test (grand N)	Test (petit N / non normal)
Moyenne diffère entre 2 groupes ?	Continue ~ Catégorielle (2)	t-test	Wilcoxon
Moyenne diffère entre 3+ groupes ?	Continue ~ Catégorielle (3+)	ANOVA	Kruskal-Wallis
Lien entre 2 variables catégorielles ?	Catégorielle x Catégorielle	Chi2	Fisher exact
Tendance croissante dans des proportions ?	Ordonnée x Binaire	Chi2 de tendance	—

> **Paramétrique vs non paramétrique** : - **Paramétrique** (t-test, ANOVA) : suppose une distribution normale. Plus puissant si l'hypothèse est respectée. - **Non paramétrique** (Wilcoxon, Kruskal-Wallis) : compare les **rangs**, pas les valeurs. Résiste aux valeurs extrêmes et aux distributions asymétriques. Plus robuste, légèrement moins puissant. - En pratique en thèse : si l'échantillon est petit (< 30) ou la distribution est asymétrique -> non paramétrique.

Syntaxe en R

```r
# t-test : âge diffère entre H et F ?
t.test(age ~ sexe, data = patients)
```

```r
# Wilcoxon : version robuste du t-test
wilcox.test(age ~ sexe, data = patients)

# Kruskal-Wallis : DFG diffère entre vert/orange/rouge ?
kruskal.test(dfg ~ risque_pr, data = patients)

# Chi2 : lien entre sexe et risque ?
chisq.test(patients$sexe, patients$risque_pr)

# Fisher : version robuste du Chi2 (petits effectifs)
fisher.test(patients$sexe, patients$risque_pr)
```

Lire les résultats

- **p < 0.05** -> différence significative -> on rejette H0
- **p >= 0.05** -> pas de différence significative -> on ne peut pas conclure
- La syntaxe `y ~ x` = "y en fonction de x"
- Fisher = plus fiable que Chi2 quand les effectifs attendus sont < 5 par case
- `add_p()` de gtsummary choisit automatiquement le bon test

Tests post-hoc — quels groupes diffèrent ?

ANOVA ou Kruskal-Wallis dit "il y a une différence quelque part entre les groupes" mais **pas lesquels**. Pour savoir quels groupes diffèrent entre eux -> test post-hoc :

```r
# Après un Kruskal-Wallis significatif :
pairwise.wilcox.test(patients$dfg, patients$risque_pr, p.adjust.method = "BH")

# Après une ANOVA significative :
pairwise.t.test(patients$dfg, patients$risque_pr, p.adjust.method = "BH")
```

- `p.adjust.method = "BH"` : correction de Benjamini-Hochberg pour les comparaisons multiples
- Le résultat = matrice de p-values entre chaque paire de groupes
- Alternative plus complète : `TukeyHSD(aov(dfg ~ risque_pr, data = patients))` après une ANOVA

Vérifier la normalité avant un test paramétrique

Avant un t-test ou une ANOVA, vérifier que les données suivent une loi normale :

```r
shapiro.test(patients$age)
# p >= 0.05 -> normalité non rejetée -> test paramétrique OK
# p < 0.05 -> distribution non normale -> test non paramétrique
```

Vérification visuelle avec un **QQ-plot** (les points doivent suivre la droite) :

```r
qqnorm(patients$age)
qqline(patients$age)
```

Limite de Shapiro-Wilk : avec un grand échantillon (n > 200-300), il rejette presque toujours la normalité, même pour des écarts négligeables en pratique. Pour les grands n, **privilégier l'inspection visuelle** (QQ-plot, histogramme).

Théorème central limite : si n > 30, la moyenne d'échantillon suit approximativement une loi normale, ce qui permet souvent d'utiliser un test paramétrique. Mais attention : pour des distributions très asymétriques (durées de séjour, revenus, données de comptage), n > 30 ne suffit pas toujours. Vérifier visuellement.

Comparaisons multiples

Quand tu fais **plusieurs tests** sur les mêmes données, le risque de faux positif augmente. Avec 20 tests à alpha = 5%, tu as ~64% de chance d'avoir au moins 1 faux positif.

```r
p_values <- c(0.03, 0.12, 0.04, 0.001, 0.08)

# Correction de Bonferroni (stricte --- divise alpha par le nombre de tests)
p.adjust(p_values, method = "bonferroni")

# Correction de Benjamini-Hochberg (recommandée --- contrôle le taux de faux positifs)
p.adjust(p_values, method = "BH")
```

Dans la thèse Predict-R : préciser que les analyses en sous-groupes sont **exploratoires** et mentionner si une correction a été appliquée.

OR vs Risque Relatif (RR)

	OR	**RR**
Formule	odds exposés / odds non-exposés	risque exposés / risque non-exposés
Quand	Cas-témoins, régression logistique	Cohorte, essais
Interprétation	Approxime le RR si événement rare (< 10%)	Directement interprétable
Piège	Surestime l'association si événement fréquent	Pas calculable en cas-témoins

Pour Predict-R : étude transversale -> on calcule des OR (pas des RR). On peut dire "les patients avec DFG < 60 ont un OR de 3.2 d'être classés à risque élevé".

Nuance importante : en transversale, l'OR est un **rapport de cotes de prévalence**, pas un vrai OR étiologique. Si l'outcome est fréquent (> 10%), l'OR surestime l'association par rapport au RR. Alternative possible : les **Prevalence Ratios** via régression de Poisson modifiée (package `logbin`). Dans la thèse Predict-R, le mentionner en discussion si la proportion de risque élevé dépasse 10%.

---

17. Régression logistique

La régression logistique permet de dire **"tel facteur augmente le risque de X, indépendamment des autres facteurs"**.

Vocabulaire

Terme	Signification
Variable à expliquer (Y)	Binaire (0/1) — ex: MRC oui/non, risque élevé oui/non
Variables explicatives (X)	Ce qui peut influencer Y — ex: âge, sexe, DFG
Odds Ratio (OR)	Le résultat principal. OR=2 -> risque doublé. OR=0.5 -> risque divisé par 2
IC 95%	Intervalle de confiance. Si contient 1 -> non significatif
Logit	ln(p/(1-p)) — transformation mathématique qui permet le modèle
Odds	Probabilité que ça arrive / probabilité que ça n'arrive pas

Pourquoi "logistique" ?

Le modèle utilise la fonction **logistique** (courbe en S) pour transformer une combinaison linéaire de variables en une probabilité entre 0 et 1. Sans cette transformation, le modèle pourrait prédire des probabilités < 0 ou > 1, ce qui n'a pas de sens.

Pourquoi "binomial" ?

`family = binomial` parce que la variable Y suit une **loi binomiale** : chaque patient a 2 issues possibles (malade/pas malade, 0/1). C'est la même loi que le pile ou face.

Lire un OR

- **OR = 1** -> pas d'effet
- **OR > 1** -> augmente le risque (facteur de risque)
- **OR < 1** -> diminue le risque (facteur protecteur)
- **IC 95% contient 1** -> non significatif (on ne peut pas conclure)
- p-value et IC sont toujours cohérents : si $p < 0.05$, alors IC ne contient pas 1

Univariable vs Multivariable

- **OR univariable** : chaque variable testée **seule** -> peut être trompeur (facteurs de confusion)
- **OR multivariable** : toutes les variables testées **en même temps** -> ajusté -> c'est ce qu'on publie

Exemple clinique : Les hommes ont plus de MRC (OR univariable = 2.0). Mais quand on ajuste sur l'âge (OR multivariable = 1.1), c'est parce que les hommes de l'échantillon sont plus vieux. L'âge est un **facteur de confusion** — il influence à la fois le sexe (dans l'échantillon) et la MRC.

Étape 1 — Créer la variable binaire Y

```
patients <- patients |>
  mutate(risque_eleve = ifelse(risque_pr == "rouge", 1, 0))

patients |> count(risque_eleve)    # vérifier : combien de 0 et de 1
```

Étape 2 — Lancer la régression

```
modele <- glm(risque_eleve ~ age + sexe + dfg,
              data = patients,
              family = binomial)
summary(modele)
```

- `glm()` : Generalized Linear Model
- `Y ~ X1 + X2 + X3` : "Y en fonction de X1, X2 et X3"
- `family = binomial` : régression **logistique** (Y binaire)
- `family = gaussian` : régression linéaire classique (Y continue)
- `family = poisson` : pour des comptages (0, 1, 2, 3...)

Lire le summary()

Le `summary(modele)` affiche pour chaque variable : - **Estimate** (beta) : le coefficient en logit -> pas interprétable directement - **Std. Error** : l'erreur standard du coefficient - **z value** : Estimate / Std. Error -> mesure à quel point le coefficient est loin de 0 - **Pr(>|z|)** : la p-value -> significatif si < 0.05

Pour obtenir les OR : `exp(coef(modele))` -> transforme les beta en OR.

Étape 3 — Tableau publication-ready avec finalfit

```
library(finalfit)    # install.packages("finalfit") si première fois

patients |>
  mutate(risque_eleve = factor(risque_eleve)) |>
  finalfit("risque_eleve", c("age", "sexe", "dfg"))
```

- `factor()` : convertit 0/1 en catégories (finalfit en a besoin)
- Donne automatiquement : descriptif par groupe + OR univariable + OR multivariable + IC 95% + p-values

Étape 4 — Forest plot (visualiser les OR)

```r
patients |>
  mutate(risque_eleve = factor(risque_eleve)) |>
  or_plot("risque_eleve", c("age", "sexe", "dfg"))
```

- Chaque point = un OR
- Chaque barre horizontale = IC 95%
- Ligne verticale à 1 = pas d'effet (référence)
- Si la barre croise la ligne 1 -> non significatif
- Axe X en échelle logarithmique (standard pour les OR)

Les maths derrière (résumé)

```
Modèle :   logit(p) = beta0 + beta1 x age + beta2 x sexeM + beta3 x dfg

Avec :     logit(p) = ln(p / (1-p))     où p = probabilité de l'événement
           odds     = p / (1-p)           ex: p=0.75 -> odds = 3 (3 contre 1)
           OR       = e^beta                 exponentielle du coefficient
           IC 95%   = e^(beta +/- 1.96 x SE)  SE = erreur standard
```

- Les `Estimate` du `summary()` = les beta (en logit)
- `finalfit` fait la conversion e^beta -> OR automatiquement
- L'IC 95% suit la même logique : on exponentie les bornes de l'IC du beta

Vérifier le modèle logistique

Deux vérifications importantes avant de publier les résultats d'une régression :

1. **Multicolinéarité** : les variables explicatives sont-elles trop corrélées entre elles ?

```r
# install.packages("car")
library(car)
vif(modele)    # VIF > 5 -> problème, retirer une variable
```

Si VIF > 5, deux variables mesurent presque la même chose. En garder une seule.

2. **Courbe ROC** : le modèle discrimine-t-il bien les cas des non-cas ?

```r
# install.packages("pROC")
library(pROC)
predictions <- predict(modele, type = "response")
roc_result <- roc(patients$risque_eleve, predictions)
auc(roc_result)        # 0.5 = hasard, 0.7-0.8 = acceptable, > 0.8 = bon
plot(roc_result)       # visualiser la courbe ROC

# IC 95% de l'AUC (pour la thèse)
ci.auc(roc_result)
```

- **AUC** (Area Under the Curve) = capacité du modèle à distinguer les cas des non-cas
- Rapporter dans la thèse : "L'AUC du modèle était de 0.78 (IC 95% : 0.71 - 0.85)"
- En entretien, on te demandera : "Comment évaluez-vous la performance de votre modèle ?" -> répondre AUC + IC 95%

3. Test de Hosmer-Lemeshow : le modèle est-il bien calibré (les probabilités prédites correspondent-elles à la réalité) ?

```
# install.packages("ResourceSelection")
library(ResourceSelection)
hoslem.test(modele$y, fitted(modele), g = 10)
# p >= 0.05 -> bonne calibration (les prédictions collent aux données)
# p < 0.05 -> mauvaise calibration
```

- **Discrimination** (ROC/AUC) = le modèle sépare-t-il bien les cas des non-cas ?
- **Calibration** (Hosmer-Lemeshow) = les probabilités prédites sont-elles fiables ?
- Dans la thèse, rapporter les deux.

PARTIE III — ANALYSES PREDICT-R (Tutoriels complets)

> **Ces tutoriels utilisent des données simulées.** Quand les vraies données seront disponibles (mai 2026), il suffira de remplacer les données simulées par le vrai CSV et d'adapter les noms de colonnes.

---

18. Données simulées Predict-R

Avant de lancer n'importe quelle analyse Predict-R, créer le jeu de données simulées :

```
library(tidyverse)
library(gtsummary)

set.seed(42)        # fixe le hasard -> même résultat à chaque exécution
n <- 150            # nombre de patients simulés

predict_r <- data.frame(
  patient_id = 1:n,
  age = round(rnorm(n, mean = 65, sd = 12)),
  sexe = sample(c("Homme", "Femme"), n, replace = TRUE),
  risque_pr = sample(c("vert", "orange", "rouge"), n,
                     replace = TRUE, prob = c(0.5, 0.3, 0.2)),
  mrc_connue = sample(c("Oui", "Non"), n,
                      replace = TRUE, prob = c(0.3, 0.7)),
  depiste_mg = sample(c(0, 1), n,
```

```
                        replace = TRUE, prob = c(0.3, 0.7)),
  nb_fdr_has = sample(0:5, n, replace = TRUE)
)
```

Explication des fonctions de simulation : - `set.seed(42)` : initialise le générateur aléatoire. Avec le même seed, tu obtiens les mêmes données à chaque fois -> reproductibilité - `rnorm(n, mean, sd)` : génère n nombres suivant une loi normale (cloche de Gauss) — ici des âges autour de 65 ans - `round()` : arrondit à l'entier (pas d'âge 65.3) - `sample(valeurs, n, replace = TRUE, prob = ...)` : tire au sort n valeurs parmi les options, avec remise, selon les probabilités données - `1:n` : crée la séquence 1, 2, 3, . . . , 150

---

19. CJP — Proportion rattrapée

Contexte

Le **critère de jugement principal** (CJP) de Predict-R : quelle proportion de patients à risque ont été **rattrapés** par le questionnaire (identifiés comme à risque par Predict-R alors que le MG ne les avait pas encore dépistés) ?

Le concept mathématique

Une **proportion** = nombre de cas / nombre total. C'est un nombre entre 0 et 1 (ou 0% et 100%).

L'**intervalle de confiance** (IC) dit : "la vraie proportion dans la population est probablement entre X% et Y%".

> **Interprétation rigoureuse de l'IC 95%** : "Si on répétait l'étude 100 fois, 95 des IC construits contiendraient la vraie valeur." Ce n'est PAS "95% de chance que la vraie valeur soit dedans" (nuance importante en entretien et dans la thèse — voir Annexe G).

Étape 1 — Identifier les patients rattrapés

```
# Un patient est "rattrapé" si :
# - Predict-R dit risque (orange ou rouge) ET le MG ne l'avait pas dépisté
predict_r <- predict_r |>
  mutate(rattrape = ifelse(risque_pr != "vert" & depiste_mg == 0, 1, 0))

# Vérifier
predict_r |> count(rattrape)
```

- `!=` : "différent de" (donc orange ou rouge)
- `&` : ET logique (les deux conditions doivent être vraies)
- `ifelse(condition, 1, 0)` : 1 si rattrapé, 0 sinon

Étape 2 — Calculer la proportion et l'IC Wilson

```r
n_rattrape <- sum(predict_r$rattrape)       # nombre de rattrapés
n_total <- nrow(predict_r)                   # nombre total de patients
proportion <- n_rattrape / n_total           # proportion brute

# IC 95% de Wilson
prop.test(n_rattrape, n_total, correct = FALSE)
```

- `prop.test()` : teste une proportion et donne l'IC
- `correct = FALSE` : désactive la correction de continuité de Yates (recommandé pour l'IC Wilson — la correction rend l'IC trop conservateur pour les proportions)

> **Pourquoi Wilson et pas un autre IC ?** Il existe plusieurs méthodes pour calculer l'IC d'une proportion : - **Wald** : la plus simple (p +/- 1.96 x SE), mais mauvaise quand p est proche de 0 ou 1 - **Wilson** : meilleure couverture, recommandée par la plupart des guides (c'est ce que `prop.test` donne) - **Clopper-Pearson** : exact, très conservateur (IC trop large) Pour une thèse, Wilson est le choix standard.

Étape 3 — Test binomial unilatéral

```r
# H0 : p <= 10% (Predict-R ne rattrape pas plus de 10% des patients)
# H1 : p > 10% (Predict-R rattrape plus de 10%)
binom.test(n_rattrape, n_total, p = 0.10, alternative = "greater")
```

Décortiquons chaque argument : - `n_rattrape` : nombre de succès (patients rattrapés) - `n_total` : nombre d'essais (total de patients) - `p = 0.10` : la proportion sous H0 (10% = le seuil qu'on veut dépasser) - `alternative = "greater"` : test **unilatéral** -> on teste seulement si c'est **supérieur** à 10%

> **Unilatéral vs bilatéral :** - **Bilatéral** (`alternative = "two.sided"`) : "est-ce que p != 10% ?" (dans les deux sens) - **Unilatéral** (`alternative = "greater"`) : "est-ce que p > 10% ?" (dans un seul sens) - On choisit unilatéral quand la question clinique a un sens : on veut prouver que Predict-R rattrape **plus** de 10%, pas juste qu'il est différent de 10% - Le choix doit être fait **avant** de regarder les données (dans le protocole)

> **Pourquoi "binomial" ?** Chaque patient est soit rattrapé (1) soit non (0) — comme un pile ou face. La somme de ces 0/1 suit une loi binomiale. Le test binomial compare exactement la proportion observée à la proportion théorique.

Interprétation : - Si p-value < 0.05 -> on rejette H0 -> Predict-R rattrape significativement plus de 10% des patients - Si p-value >= 0.05 -> on ne peut pas conclure que Predict-R rattrape plus de 10%

> **ATTENTION — P-hacking** : changer `p = 0.10` en `p = 0.09` après avoir vu les résultats pour obtenir p < 0.05, c'est de la fraude scientifique. Le seuil est défini dans le protocole AVANT le recueil.

---

20. Kappa de Cohen — Concordance

Contexte

Le **Kappa de Cohen** mesure la concordance entre deux évaluateurs (ou deux méthodes). Dans Predict-R : est-ce que le questionnaire et le MG identifient les **mêmes** patients comme étant à risque ?

Le concept mathématique

Le Kappa corrige la concordance observée par la concordance attendue **par hasard** :

```
kappa = (concordance observée - concordance due au hasard) / (1 - concordance due au hasard)
```

- Si deux évaluateurs sont d'accord 80% du temps, mais que par hasard on attendrait 60% d'accord, alors kappa = (0.80 - 0.60) / (1 - 0.60) = 0.50

Échelle d'interprétation (Landis & Koch, 1977)

Kappa	Interprétation
< 0	Désaccord (pire que le hasard)
0 - 0.20	Accord faible
0.21 - 0.40	Accord passable
0.41 - 0.60	Accord modéré
0.61 - 0.80	Accord substantiel
0.81 - 1.00	Accord presque parfait

Le code

```r
library(irr)     # install.packages("irr") si première fois

# Créer les variables binaires (risque oui/non pour chaque méthode)
predict_r <- predict_r |>
  mutate(risque_binaire = ifelse(risque_pr != "vert", 1, 0))

# Calculer le Kappa
kappa_result <- kappa2(data.frame(predict_r$risque_binaire,
                                  predict_r$depiste_mg))
kappa_result
```

Explication : - `library(irr)` : package **I**nter-**R**ater **R**eliability (fiabilité inter-évaluateurs) - `risque_binaire` : Predict-R dit risque (1) ou pas (0) - `depiste_mg` : le MG a dépisté (1) ou pas (0) - `kappa2()` : calcule le Kappa de Cohen pour 2 évaluateurs - `data.frame(colonne1, colonne2)` : met les 2 colonnes côte à côte dans un tableau

Résultat : Le Kappa, sa z-value et sa p-value. Si $p < 0.05$, la concordance est significativement différente du hasard.

Kappa pondéré — pour variables ordinales

Si tu gardes les 3 niveaux (vert/orange/rouge) au lieu de binariser, utiliser le **Kappa pondéré** qui tient compte de la distance entre les désaccords (confondre vert et orange est moins grave que confondre vert et rouge) :

```r
# Kappa pondéré (weighted kappa)
kappa2(data.frame(variable1, variable2), weight = "squared")
# weight = "squared" : pondération quadratique (standard)
```

À discuter dans la thèse : justifier le choix entre Kappa simple (binaire) et Kappa pondéré (ordinal).

> **IC du Kappa** : `kappa2()` ne donne pas directement l'IC 95% du Kappa. Pour la thèse, utiliser `psych::cohen.kappa()` qui fournit le Kappa **avec** son IC 95% — un rapporteur de thèse le demandera.
>
> ```r
> # install.packages("psych")
> psych::cohen.kappa(table(variable1, variable2))
> ```

---

21. Se, Sp, VPP, VPN — Performance diagnostique

Contexte

Si on a un **gold standard** (vrai diagnostic de MRC), on peut évaluer la performance de Predict-R comme outil de dépistage.

Le cours : le tableau 2x2

Tout repose sur un tableau à 4 cases :

```
                    Malade (gold standard)
                    Oui           Non
Test    Positif     VP (Vrai +)   FP (Faux +)
        Négatif     FN (Faux -)   VN (Vrai -)
```

Indicateur	Formule	Question	Interprétation
Sensibilité (Se)	VP / (VP + FN)	Parmi les malades, combien sont détectés ?	Se = 90% -> le test détecte 90% des malades
Spécificité (Sp)	VN / (VN + FP)	Parmi les sains, combien sont correctement exclus ?	Sp = 80% -> le test exclut correctement 80% des sains

Indicateur	Formule	Question	Interprétation
VPP	VP / (VP + FP)	Si le test est +, quelle proba d'être malade ?	VPP = 70% -> 70% des tests + sont de vrais malades
VPN	VN / (VN + FN)	Si le test est -, quelle proba d'être sain ?	VPN = 95% -> 95% des tests - sont vraiment sains

> **Point clé** : La Se et la Sp sont des propriétés **intrinsèques** du test (elles ne changent pas avec la prévalence). La VPP et la VPN dépendent de la **prévalence** de la maladie dans la population. Dans une population où la maladie est rare, même un bon test aura une VPP basse (beaucoup de faux positifs par rapport aux vrais positifs).

Le code

```r
library(epiR)     # install.packages("epiR") si première fois

# Simuler un gold standard (vrai diagnostic MRC)
predict_r <- predict_r |>
  mutate(
    mrc_vraie = sample(c(0, 1), n(), replace = TRUE, prob = c(0.6, 0.4)),
    test_positif = ifelse(risque_pr != "vert", 1, 0)
  )

# Créer le tableau 2x2
tab <- table(predict_r$test_positif, predict_r$mrc_vraie)

# IMPORTANT : réordonner pour que epi.tests fonctionne
# epi.tests attend : Test+ / Malade+ en haut à gauche
tab <- tab[c(2,1), c(2,1)]
tab     # vérifier : la case en haut à gauche = VP

# Calculer Se, Sp, VPP, VPN
epi.tests(tab)
```

Explication du réordonnancement : - `table()` met 0 avant 1 par défaut -> la case en haut à gauche = test- / malade- - `epi.tests()` attend test+ / malade+ en haut à gauche (les VP) - `tab[c(2,1), c(2,1)]` : inverse les lignes (2 avant 1) et les colonnes (2 avant 1). La syntaxe `tableau[lignes, colonnes]` permet de sélectionner des parties d'un tableau. `c(2,1)` = "prends la ligne 2 puis la ligne 1" -> inverse l'ordre - **Règle** : toujours le **test en lignes** et le **malade en colonnes**, avec le positif en premier

---

22. Taux de participation et complétion

Contexte

Avant les analyses, rapporter combien de patients ont participé et combien ont complété le questionnaire en entier.

Le code

```r
# Simuler les données
n_eligibles <- 200       # patients éligibles
n_inclus <- 150          # patients ayant accepté de participer
n_complets <- 142        # patients ayant terminé le questionnaire

# Taux de participation
taux_participation <- n_inclus / n_eligibles
taux_participation     # 0.75 = 75%

# IC 95% du taux de participation
prop.test(n_inclus, n_eligibles, correct = FALSE)

# Taux de complétion
taux_completion <- n_complets / n_inclus
taux_completion        # 0.947 = 94.7%

# IC 95% du taux de complétion
prop.test(n_complets, n_inclus, correct = FALSE)
```

C'est le même `prop.test()` que pour le CJP — on teste une proportion et on obtient son IC Wilson.

---

23. Satisfaction — Échelle de Likert

Contexte

Les patients notent leur satisfaction sur une échelle de 1 à 5 (1 = pas du tout d'accord, 5 = tout à fait d'accord). On rapporte la **médiane et l'IQR** (pas la moyenne — une échelle ordinale n'est pas continue).

> **Ordinal vs continu :** - **Ordinal** : les valeurs ont un ordre ($1 < 2 < 3$) mais l'écart entre 1 et 2 n'est pas forcément le même qu'entre 4 et 5 - **Continu** : les écarts sont égaux (la température, l'âge, le DFG) - Pour les données ordinales -> médiane + IQR (pas la moyenne)

Le code

```r
# Simuler les données de satisfaction
satisfaction <- data.frame(
  patient_id = 1:150,
  facilite = sample(1:5, 150, replace = TRUE, prob = c(0.05, 0.1, 0.2, 0.3, 0.35)),
  utilite = sample(1:5, 150, replace = TRUE, prob = c(0.03, 0.07, 0.15, 0.35, 0.40)),
  recommande = sample(1:5, 150, replace = TRUE, prob = c(0.05, 0.10, 0.20, 0.30, 0.35))
)

# Tableau avec médiane (Q1, Q3)
satisfaction |>
```

```r
  select(facilite, utilite, recommande) |>
  tbl_summary(
    type = list(everything() ~ "continuous"),
    statistic = all_continuous() ~ "{median} ({p25}, {p75})",
    label = list(
      facilite ~ "Facilite d'utilisation",
      utilite ~ "Utilite percue",
      recommande ~ "Recommanderait l'outil"
    )
  )
```

Point crucial : `type = list(everything() ~ "continuous")`. Sans cette ligne, gtsummary voit des valeurs 1, 2, 3, 4, 5 et les traite comme des **catégories** (affiche n et %). Avec `"continuous"`, il calcule la médiane et les quartiles.

Interprétation

- Médiane 4 (3, 5) -> la moitié des patients ont noté >= 4, 75% ont noté >= 3
- Si médiane >= 4 -> satisfaction globalement positive

---

24. Analyses en sous-groupes

Contexte

Comparer les caractéristiques des patients selon un sous-groupe (MRC connue vs inconnue, par tranche d'âge, etc.). C'est un Tableau 1 stratifié.

Préparer les tranches d'âge

```r
predict_r <- predict_r |>
  mutate(
    tranche_age = case_when(
      age < 50 ~ "<50 ans",
      age < 65 ~ "50-64 ans",
      age < 75 ~ "65-74 ans",
      TRUE ~ ">=75 ans"
    ),
    tranche_age = factor(tranche_age,
                         levels = c("<50 ans", "50-64 ans", "65-74 ans", ">=75 ans"))
  )
```

- `case_when()` crée les catégories, `TRUE ~ ">=75 ans"` = le "sinon" (tout le reste)
- `factor(..., levels = ...)` : force l'ordre des catégories (sinon R trie par alphabet)

Sous-groupe 1 : MRC connue vs inconnue

```r
predict_r |>
  select(age, sexe, risque_pr, nb_fdr_has, mrc_connue) |>
  tbl_summary(
    by = mrc_connue,
    statistic = list(
      all_continuous() ~ "{mean} ({sd})",
      all_categorical() ~ "{n} ({p}%)"
    ),
    label = list(
      age ~ "Age (annees)",
      sexe ~ "Sexe",
      risque_pr ~ "Niveau de risque Predict-R",
      nb_fdr_has ~ "Nombre de FDR HAS"
    )
  ) |>
  add_p() |>
  add_overall()
```

Sous-groupe 2 : par tranche d'âge

```r
predict_r |>
  select(sexe, risque_pr, nb_fdr_has, mrc_connue, tranche_age) |>
  tbl_summary(
    by = tranche_age,
    type = list(nb_fdr_has ~ "continuous"),
    statistic = list(
      all_continuous() ~ "{mean} ({sd})",
      all_categorical() ~ "{n} ({p}%)"
    ),
    label = list(
      sexe ~ "Sexe",
      risque_pr ~ "Niveau de risque Predict-R",
      nb_fdr_has ~ "Nombre de FDR HAS",
      mrc_connue ~ "MRC connue"
    )
  ) |>
  add_p()
```

Note : on ne met pas `age` dans le tableau quand on stratifie par tranche d'âge (pas de sens). On force `nb_fdr_has ~ "continuous"` car avec 4 tranches d'âge x 6 valeurs, Fisher peut planter (tableau croisé trop grand).

Interprétation d'un tableau par sous-groupes

- La p-value compare **tous les groupes en une seule fois** (pas 2 à 2)
- $p < 0.05$ dit "il y a une différence quelque part" mais ne dit pas entre quels groupes
- Dans la thèse : citer la p-value globale, puis illustrer avec les groupes extrêmes

---

25. Chi2 de tendance — Cochran-Armitage

Contexte

Quand les catégories ont un **ordre** (vert < orange < rouge), on veut tester s'il y a une **tendance** : est-ce que la proportion de MRC connue **augmente** avec le niveau de risque ?

Chi2 classique vs Chi2 de tendance

	Chi2 classique	Chi2 de tendance
Question	Y a-t-il une différence entre les groupes ?	Y a-t-il une tendance croissante/décroissante ?
df	k-1 (ici 2)	toujours 1
Puissance	Moins puissant pour détecter une tendance	Plus puissant si la tendance est linéaire
Condition	Catégories quelconques	Catégories **ordonnées**

> **Note statistique** : le Chi2 de tendance a un seul degré de liberté (df = 1) car il teste un seul paramètre : la pente de la tendance. Le Chi2 classique avec 3 groupes a df = 2 car il teste la différence entre chaque paire. Avec moins de df, le test est plus puissant pour détecter ce qu'il cherche.

Le code

```r
# Ordonner les niveaux de risque (CRUCIAL)
predict_r <- predict_r |>
  mutate(risque_pr = factor(risque_pr, levels = c("vert", "orange", "rouge")))

# Tableau croisé : risque x MRC connue
tab_tendance <- table(predict_r$risque_pr, predict_r$mrc_connue)
tab_tendance    # vérifier

# Chi2 de tendance (Cochran-Armitage)
prop.trend.test(
  x = tab_tendance[, "Oui"],      # nombre de "Oui" par niveau de risque
  n = rowSums(tab_tendance)         # total par niveau de risque
)
```

Explication : - `factor(..., levels = c("vert", "orange", "rouge"))` : force l'ordre. Sans ça, R trie par alphabet (orange, rouge, vert) et la tendance n'a pas de sens - `table(risque_pr, mrc_connue)` : tableau croisé, lignes = risque, colonnes = MRC - `tab_tendance[, "Oui"]` : extrait la colonne "Oui" = nombre de MRC connue par niveau - `rowSums(tab_tendance)` : total de patients par niveau de risque - `prop.trend.test(x, n)` : teste si la proportion x/n augmente linéairement - R assigne automatiquement les scores 1, 2, 3 aux niveaux ordonnés

Résultat : `X-squared`, `df = 1`, `p-value` - p < 0.05 -> tendance significative (la proportion augmente/diminue avec le niveau de risque) - p >= 0.05 -> pas de tendance significative

Analyse de sensibilité — vérifier la robustesse des résultats

L'analyse de sensibilité consiste à **refaire la même analyse en changeant une hypothèse** pour voir si les résultats tiennent. C'est attendu dans une thèse et en entretien.

Exemples pour Predict-R :

```r
# 1. Exclure les patients avec MRC connue et recalculer le CJP
predict_r_inconnus <- predict_r |> filter(mrc_connue == 0)
prop.test(sum(predict_r_inconnus$rattrape),
          nrow(predict_r_inconnus), p = 0.10,
          alternative = "greater", correct = FALSE)

# 2. Recalculer le Kappa en excluant les patients > 80 ans
predict_r_jeunes <- predict_r |> filter(age <= 80)
kappa2(data.frame(predict_r_jeunes$risque_binaire,
                  predict_r_jeunes$depiste_mg))

# 3. Modifier le seuil de positivité (orange+rouge vs rouge seul)
predict_r <- predict_r |>
  mutate(risque_strict = ifelse(risque_pr == "rouge", 1, 0))
```

En entretien : "J'ai vérifié que mes résultats sont robustes en excluant les patients avec MRC connue — le CJP reste significatif." Montre que tu es rigoureux.

PARTIE IV — OUTILS COMPLÉMENTAIRES (Aperçu Phase 3)

> **Ces sections sont des introductions.** Tu n'as pas encore pratiqué ces outils. Elles te donnent le vocabulaire et les premiers pas pour que tu ne partes pas de zéro quand tu les aborderas.

---

26. Git — Contrôle de version

Git enregistre l'historique de tes fichiers. Tu peux revenir en arrière, travailler à plusieurs, et montrer ton travail sur GitHub/GitLab.

Vocabulaire

Terme	Signification
Repository (repo)	Un dossier suivi par Git
Commit	Un point de sauvegarde (comme une photo de ton code à un instant)

Terme	Signification
Branch	Une version parallèle (pour tester sans casser le code principal)
Push	Envoyer tes commits vers GitHub/GitLab
Pull	Récupérer les changements depuis GitHub/GitLab
Merge	Fusionner une branche dans une autre
.gitignore	Fichier qui liste ce que Git doit ignorer (données patients, .Rhistory)

Premiers pas (dans le terminal RStudio)

```
# Initialiser un repo Git dans ton projet
git init

# Voir l'état des fichiers
git status

# Ajouter des fichiers à suivre
git add script_analyse.R
git add .     # ajouter tout (attention : vérifier .gitignore d'abord)

# Créer un commit (point de sauvegarde)
git commit -m "Ajout analyse descriptive Predict-R"

# Voir l'historique
git log --oneline

# Envoyer vers GitHub (après avoir créé le repo en ligne)
git remote add origin https://github.com/ton-compte/predict-r.git
git push -u origin main
```

.gitignore pour un projet R

```
# Ne JAMAIS mettre sur GitHub :
*.csv          # données patients (confidentialité)
*.xlsx
.Rhistory
.RData
.Renviron      # peut contenir des mots de passe
```

> **SpFrance utilise GitLab**, pas GitHub. C'est la même chose sauf l'interface web. Les commandes Git sont identiques.

Ressource : Learn Git Branching — tutoriel interactif gratuit

---

27. SQL — Interroger des bases de données

SQL = le langage pour interroger des bases de données. Les concepts sont les mêmes que dplyr.

Équivalences dplyr <-> SQL

dplyr	SQL	Ce que ça fait
`filter()`	`WHERE`	Filtrer les lignes
`select()`	`SELECT`	Choisir les colonnes
`arrange()`	`ORDER BY`	Trier
`mutate()`	`SELECT ..., expression AS nom`	Créer une colonne
`group_by() + summarise()`	`GROUP BY` + `SUM/COUNT/AVG`	Agréger
`left_join()`	`LEFT JOIN ... ON`	Joindre deux tables
`count()`	`COUNT(*)`	Compter

Exemple

```sql
-- Nombre de cas par département, trié du plus au moins
SELECT departement, COUNT(*) AS nb_cas
FROM cas_grippe
WHERE date_symptomes >= '2026-01-01'
GROUP BY departement
ORDER BY nb_cas DESC;
```

Utiliser SQL depuis R

```r
# install.packages("DBI")
# install.packages("RSQLite")    # ou RPostgres pour PostgreSQL
library(DBI)

# Se connecter
con <- dbConnect(RSQLite::SQLite(), "ma_base.sqlite")

# Exécuter une requête
resultats <- dbGetQuery(con, "SELECT * FROM patients WHERE age > 65")

# Ou utiliser dplyr directement sur la base (pas besoin de SQL)
library(dbplyr)
patients_db <- tbl(con, "patients")
patients_db |> filter(age > 65) |> collect()    # collect() = ramener en R

dbDisconnect(con)
```

Ressource : SQLBolt — tutoriel interactif gratuit (12 leçons, ~2h)

---

28. Shiny — Applications web interactives

Shiny transforme un script R en application web. L'utilisateur clique sur des boutons, les graphiques se mettent à jour.

Structure d'une app Shiny

```r
library(shiny)

# Interface utilisateur (ce que l'utilisateur voit)
ui <- fluidPage(
  titlePanel("Dashboard Predict-R"),
  sidebarLayout(
    sidebarPanel(
      selectInput("variable", "Variable :", choices = c("age", "dfg")),
      selectInput("groupe", "Grouper par :", choices = c("risque_pr", "sexe"))
    ),
    mainPanel(
      plotOutput("graphique"),
      tableOutput("tableau")
    )
  )
)

# Serveur (le code R qui tourne derrière)
server <- function(input, output) {
  output$graphique <- renderPlot({
    ggplot(patients, aes(x = .data[[input$groupe]], y = .data[[input$variable]])) +
      geom_boxplot() + theme_classic()
  })
  output$tableau <- renderTable({
    patients |> group_by(.data[[input$groupe]]) |>
      summarise(moyenne = mean(.data[[input$variable]], na.rm = TRUE))
  })
}

# Lancer l'application
shinyApp(ui = ui, server = server)
```

- `ui` = l'interface (menus, boutons, zones d'affichage)
- `server` = le code R qui réagit aux choix de l'utilisateur
- `input$variable` = la valeur choisie par l'utilisateur dans le menu
- `output$graphique` = le graphique qui se met à jour

Publier en ligne

```r
# install.packages("rsconnect")
library(rsconnect)
deployApp()    # envoie sur shinyapps.io (gratuit, 5 apps, 25h/mois)
```

Ressource : Shiny — Get Started — tutoriel officiel

---

29. Cartographie — Visualiser des données géographiques

En ARS, on cartographie des taux par département ou commune.

Premiers pas avec sf + ggplot2

```
# install.packages("sf")
library(sf)
library(tidyverse)

# Charger un fond de carte (shapefile des départements français)
# Télécharger sur : https://www.data.gouv.fr/fr/datasets/contours-des-departements/
departements <- st_read("departements.geojson")

# Joindre tes données au fond de carte
departements <- departements |>
  left_join(tes_donnees, by = "code_departement")

# Carte choroplèthe (couleur = intensité d'un indicateur)
ggplot(departements) +
  geom_sf(aes(fill = taux_incidence)) +
  scale_fill_gradient(low = "white", high = "red") +
  labs(title = "Taux d'incidence par departement", fill = "Taux") +
  theme_void()
```

- `st_read()` : lit un fichier géographique (GeoJSON, Shapefile)
- `geom_sf()` : dessine les polygones (départements, communes)
- `scale_fill_gradient()` : dégradé de couleurs
- `theme_void()` : supprime les axes (on veut juste la carte)

---

30. Analyse de survie — Kaplan-Meier et Cox

L'analyse de survie modélise le **temps jusqu'à un événement** (décès, rechute, guérison). Pas directement utile pour Predict-R (transversale) mais demandé à SpFrance.

Kaplan-Meier — courbe de survie

```
# install.packages("survival")
# install.packages("survminer")    # pour les graphiques
library(survival)
library(survminer)
```

```r
# Créer un objet de survie
surv_obj <- Surv(time = patients$duree_suivi, event = patients$deces)

# Estimer la survie
fit <- survfit(surv_obj ~ groupe, data = patients)

# Courbe de Kaplan-Meier
ggsurvplot(fit, data = patients, pval = TRUE, risk.table = TRUE)
```

Cox — régression sur le temps de survie

```r
# Modèle de Cox : quels facteurs influencent la survie ?
cox <- coxph(Surv(duree_suivi, deces) ~ age + sexe + traitement, data = patients)
summary(cox)    # HR (Hazard Ratio) = comme un OR mais pour la survie
```

- **HR > 1** -> augmente le risque de décès (facteur de mauvais pronostic)
- **HR < 1** -> diminue le risque (facteur protecteur)

---

31. Reproductibilité avancée — renv

`renv` gère les **versions des packages** dans ton projet. Quand tu partages ton code, l'autre personne installe exactement les mêmes versions.

```r
# install.packages("renv")

# Initialiser renv dans un projet
renv::init()          # crée un fichier renv.lock avec les versions actuelles

# Installer un package (renv le note automatiquement)
renv::install("gtsummary")

# Sauvegarder l'état actuel des packages
renv::snapshot()      # met à jour renv.lock

# Restaurer les packages à partir du lock file (sur un autre ordinateur)
renv::restore()       # installe exactement les mêmes versions
```

> C'est le standard de reproductibilité à SpFrance. Quand quelqu'un clone ton repo GitLab et fait `renv::restore()`, il a exactement le même environnement que toi.

ANNEXES

A. Bonnes pratiques

Workflow de nettoyage de données (dans cet ordre)

Quand tu reçois un fichier brut en poste, toujours suivre cette séquence :

```r
donnees <- read_csv("fichier.csv")        # 1. Charger
donnees <- donnees |> clean_names()       # 2. Nettoyer les noms de colonnes
glimpse(donnees)                           # 3. Vérifier types et colonnes
nrow(donnees)                              # 4. Combien de lignes ?
donnees <- donnees |> distinct()          # 5. Supprimer les doublons
nrow(donnees)                              # 6. Combien de lignes après ? (comparer)
colSums(is.na(donnees))                    # 7. Compter les NA par colonne
donnees <- donnees |> filter(!is.na(id))  # 8. Filtrer les lignes invalides
donnees <- donnees |> mutate(...)         # 9. Créer les variables dérivées
```

Principes généraux

- **Script vs Console** : résultats finaux dans le script, tests dans la console
- **Stocker** : `patients <- patients |> mutate(...)` sinon le résultat est juste affiché
- **Aérer** : retour à la ligne après `+` et `,`
- **Parenthèses** : `ggplot(données, aes(...))` -> 2 parenthèses fermantes avant le `+`
- **Chemins** : toujours `/` (slash) jamais \ (backslash), toujours entre guillemets
- **NA** : toujours vérifier et rapporter les données manquantes
- **R Markdown** : un fichier = texte + code + résultats, tout reproductible
- **Protéger ses données** : quand on transforme, utiliser des noms sémantiques (`donnees_brutes`, `donnees_nettoyees`, `donnees_analyse`) — éviter `donnees2`, `donnees3` qui deviennent vite incompréhensibles
- **set.seed()** : toujours fixer le seed quand on simule des données, pour la reproductibilité
- **Commentaires** : `# commentaire` -> expliquer le pourquoi, pas le quoi

---

B. Pièges courants

Piège	Exemple faux	Solution
Oublier les guillemets	`Bonjour`	`"Bonjour"`
Majuscule/minuscule	`DFG` au lieu de `dfg`	Vérifier le nom exact
Écrire en français	`groupe_by()`	`group_by()` — R c'est en anglais
`=` vs `==`	`risque = "rouge"` pour comparer	`==` pour tester, `=` dans les fonctions
Pas de library	`filter()` introuvable	`library(tidyverse)` d'abord
Guillemet manquant	`y = "texte, color = "x"`	Vérifier chaque `"` ouvrant a son fermant
`color` vs `fill` dans labs	Warning "unknown labels"	`labs(fill = ...)` si `aes(fill = ...)`

Piège	Exemple faux	Solution
Ne pas stocker	`patients` puis `mutate(...)` sans `<-`	Ajouter `patients <-` devant pour sauvegarder
`na.rm` pas `rm.na`	`mean(x, rm.na = TRUE)`	`mean(x, na.rm = TRUE)`
Backslash dans chemins	`"C:\Users\..."`	`"C:/Users/..."` — toujours `/`
CSV introuvable	`read_csv("fichier.csv")`	Vérifier `getwd()` et `list.files()`
Oublier `()` après fonction	`glimpse` au lieu de `glimpse(donnees)`	Sans `()` R affiche le code source
`;` au lieu de `,`	`c("a"; "b")`	Toujours `,` pour séparer les arguments
`fill` fixe dans `aes()`	`aes(fill = "blue")`	Mettre dans `geom_*()` : `geom_histogram(fill = "blue")`
Confondre `+` et pipe	`slice_max() + filter()`	`+` c'est ggplot, le pipe c'est pour dplyr
Mauvais nom après summarise	`slice_max(esp)` après `summarise(esp_moyenne = ...)`	Utiliser le nom créé : `slice_max(esp_moyenne)`
`none` sans guillemets	`legend.position = none`	`legend.position = "none"`
`add_overall` sans `()`	`add_overall`	`add_overall()` — toujours les parenthèses
Fisher plante (gros tableau)	`add_p()` avec beaucoup de catégories	Forcer `type = list(var ~ "continuous")`

Messages d'erreur fréquents et solutions

Message d'erreur	Cause probable	Solution
`Error in filter(): object 'x' not found`	La variable n'existe pas dans le data frame	Vérifier le nom exact avec `names(donnees)` ou `glimpse(donnees)`
`could not find function "filter"`	Le package n'est pas chargé	`library(tidyverse)` en début de script
`unexpected ')' in "..."`	Parenthèse en trop ou mal placée	Compter les parenthèses ouvrantes et fermantes
`unexpected '}' in "..."`	Accolade en trop (souvent dans R Markdown)	Vérifier les blocs `{r}`
`object 'donnees' not found`	Le data frame n'a pas été créé ou le nom est différent	Vérifier l'onglet Environment dans RStudio
`non-numeric argument to binary operator`	On fait du calcul sur du texte	Vérifier les types avec `str()` ou `glimpse()`
`replacement has X rows, data has Y rows`	`mutate()` avec un vecteur de mauvaise longueur	Vérifier que le calcul produit une valeur par ligne
`cannot open file 'fichier.csv': No such file or directory`	Le fichier n'est pas dans le bon dossier	`getwd()` + `list.files()` pour vérifier

Message d'erreur	Cause probable	Solution
`Error in loadNamespace: there is no package called 'X'`	Package pas installé	`install.packages("X")` une fois, puis `library(X)`
`Warning: Removed N rows containing missing values`	Des NA dans les données	Normal pour ggplot — vérifier si c'est attendu
`each group size must be the same` (kappa2)	Les deux colonnes n'ont pas le même nombre de lignes	Vérifier avec `nrow()` et `sum(is.na())`

Réflexe debugging : lire le message d'erreur du bas vers le haut. La dernière ligne est souvent la plus utile. Copier-coller l'erreur dans Google ou StackOverflow marche très bien.

C. Raccourcis clavier AZERTY

Symbole	Touche
`{ }`	AltGr + 4 / AltGr + +
`[ ]`	AltGr + 5 / AltGr +)
`` ` ``	AltGr + 7
`|`	AltGr + 6
`~`	AltGr + 2
`<-`	Alt + - (dans RStudio)

D. Encodage UTF-8

- **UTF-8** = standard universel qui gère les accents (é, è, ç, ù...)
- RStudio -> Tools -> Global Options -> Code -> Saving -> **UTF-8**
- Si popup "ré-encoder" -> choisir UTF-8

E. Lien avec Predict-R — Vue d'ensemble

Ce que j'apprends	Analyse Predict-R correspondante
`group_by() + summarise()`	Tableau 1 descriptif (caractéristiques par groupe)
`gtsummary::tbl_summary()`	Tableau 1 publication-ready avec p-values
`filter()`	Sélection de sous-populations
`mutate()` + `ifelse()`	Variables dérivées (stades MRC, catégories d'âge, variable binaire)
`mean()` sur logiques	Proportions (Se, Sp, VPP, VPN)
`prop.test()`	CJP : proportion rattrapée + IC Wilson

Ce que j'apprends	Analyse Predict-R correspondante
`binom.test()`	CJP : test binomial unilatéral (H0 : p <= 10%)
`kappa2()` (irr)	Concordance Predict-R vs MG
`epi.tests()` (epiR)	Se, Sp, VPP, VPN du questionnaire
`glm(family = binomial)`	Régression logistique — facteurs associés au risque
`finalfit()` + `or_plot()`	Tableau OR + forest plot publication-ready
`prop.trend.test()`	Chi2 de tendance — proportion par niveau de risque
`tbl_summary(type = "continuous")`	Satisfaction Likert — médiane (Q1, Q3)
`tbl_summary(by = sous_groupe)`	Analyses en sous-groupes (MRC connue, tranches d'âge)
`ggplot2` tous types	Figures 1-5 de la thèse
R Markdown	Rédiger les résultats de thèse (texte + code + figures)

Packages utilisés — Résumé

Package	À quoi ça sert	Fonction principale
`tidyverse`	Manipulation et visualisation	`dplyr`, `ggplot2`, `readr`
`gtsummary`	Tableaux descriptifs publication-ready	`tbl_summary()`, `add_p()`
`finalfit`	Tableaux de régression + forest plots	`finalfit()`, `or_plot()`
`irr`	Concordance inter-évaluateurs	`kappa2()`
`epiR`	Épidémiologie (Se, Sp, VPP, VPN)	`epi.tests()`
`scales`	Formater les axes	`scales::percent`
`readxl`	Importer des fichiers Excel (.xlsx)	`read_excel()`
`janitor`	Nettoyer les noms de colonnes	`clean_names()`
`lubridate`	Manipuler les dates	`dmy()`, `floor_date()`
`car`	Diagnostics de régression	`vif()`
`pROC`	Courbe ROC et AUC	`roc()`, `auc()`, `ci.auc()`
`pwr`	Calcul de puissance statistique	`pwr.p.test()`
`here`	Chemins relatifs au projet	`here()`
`flextable`	Tableaux vers Word	`as_flex_table()`, `save_as_docx()`
`writexl`	Export vers Excel	`write_xlsx()`
`stringr`	Manipulation de texte (inclus tidyverse)	`str_detect()`, `str_replace()`
`purrr`	Itération fonctionnelle (inclus tidyverse)	`map()`, `map_dfr()`
`ResourceSelection`	Calibration du modèle logistique	`hoslem.test()`
`DBI` + `RSQLite`	Connexion bases de données SQL	`dbConnect()`, `dbGetQuery()`

Package	À quoi ça sert	Fonction principale
`sf`	Données géographiques et cartographie	`st_read()`, `geom_sf()`
`survival` + `survminer`	Analyse de survie	`survfit()`, `coxph()`, `ggsurvplot()`
`renv`	Gestion des versions de packages	`renv::init()`, `renv::snapshot()`
`shiny`	Applications web interactives	`shinyApp()`

F. Glossaire épidémiologique

Pour les entretiens ARS / Santé Publique France.

Mesures de fréquence

Terme	Définition	Exemple
Prévalence	Proportion de malades à un instant donné ("photo")	5% de la population a du diabète aujourd'hui
Incidence	Nombre de **nouveaux** cas sur une période ("film")	200 nouveaux cas de grippe cette semaine
Taux d'incidence	Incidence / population à risque / temps	3.5 pour 100 000 personnes-années
Létalité	Décès / nombre de malades	Grippe ~0.1%, Ebola ~50%
Mortalité	Décès / population totale / temps	9.1 pour 1 000 habitants par an

Mesures d'association

Terme	Définition	Étude
Risque Relatif (RR)	Risque exposés / risque non-exposés	Cohorte, essais
Odds Ratio (OR)	Odds exposés / odds non-exposés	Cas-témoins, régression logistique
Risque Attribuable (RA)	Risque exposés - risque non-exposés	Impact de l'exposition
NNT	1 / RA = nombre à traiter pour éviter 1 cas	Utilité clinique

Types d'études (du plus faible au plus fort niveau de preuve)

1. **Étude transversale** (Predict-R est ici) — mesure à un instant donné. Donne des prévalences et des OR. Pas de causalité.
2. **Étude cas-témoins** — part des malades, cherche les expositions passées. Rétrospective. Donne des OR.

3. **Étude de cohorte** — part des exposés, suit dans le temps. Prospective. Donne des RR et des incidences.
4. **Essai contrôlé randomisé (ECR)** — gold standard. Randomisation élimine les facteurs de confusion. Donne des RR.
5. **Méta-analyse** — synthèse de plusieurs études. Plus haut niveau de preuve.

Les 3 grands biais

Biais	Définition	Exemple	Solution
Sélection	L'échantillon ne représente pas la population	Ne recruter qu'en consultation -> manque les asymptomatiques	Recrutement exhaustif, randomisation
Classement	Erreur de mesure de l'exposition ou de la maladie	Patient qui oublie son exposition (biais de mémorisation)	Mesures objectives, questionnaires validés
Confusion	Un 3ème facteur influence l'exposition ET la maladie	Café associé au cancer du poumon -> parce que les fumeurs boivent plus de café	Ajustement (régression), stratification

Facteur de confusion

Un facteur de confusion est une variable qui : 1. Est associée à l'exposition 2. Est un facteur de risque de la maladie 3. N'est PAS une conséquence de l'exposition

C'est pour ça qu'on fait de la régression **multivariable** : pour ajuster sur les facteurs de confusion.

Critères de causalité (Bradford Hill, 1965)

1. **Force** de l'association (OR/RR élevé)
2. **Cohérence** (retrouvée dans d'autres études)
3. **Temporalité** (exposition précède la maladie — seul critère obligatoire)
4. **Gradient dose-réponse** (plus on est exposé, plus le risque augmente)
5. **Plausibilité biologique**
6. **Spécificité**, **Analogie**, **Réversibilité**

Standardisation des taux

Pourquoi ? Comparer des taux bruts entre départements ou régions est trompeur si les populations n'ont pas la même structure d'âge (un département "vieux" aura un taux de mortalité brut plus élevé, même si la santé est meilleure).

Standardisation directe : on applique les taux par âge de chaque département à une **population de référence** commune (ex : population France entière). Résultat = "si tous les départements avaient la même pyramide d'âge, quel serait leur taux ?"

Standardisation indirecte (SMR/SIR) : - **SMR** (Ratio Standardisé de Mortalité) = décès observés / décès attendus - SMR = 1.2 -> 20% de surmortalité par rapport à la référence - SMR < 1 -> sous-mortalité, SMR > 1 -> surmortalité - Utilisé quand les effectifs par âge sont petits

C'est le **pain quotidien** de l'ARS et de SpFrance. En entretien : "Comment compareriez-vous la mortalité entre deux départements ?" -> répondre standardisation.

```r
# Exemple simplifié de standardisation directe
# (en pratique, utiliser le package epitools ou PHEindicatormethods)
taux_ajuste <- sum(taux_par_age * pop_reference_par_age) / sum(pop_reference_par_age)
```

Courbe épidémique

La **courbe épi** = histogramme du nombre de cas par unité de temps (jour, semaine). C'est le **premier réflexe** en surveillance. En R :

```r
library(tidyverse)
library(lubridate)

# Données de cas avec date de symptômes
cas <- data.frame(
  date_symptomes = dmy(c("01/03/2026", "02/03/2026", "02/03/2026",
                         "05/03/2026", "07/03/2026", "08/03/2026",
                         "08/03/2026", "08/03/2026", "10/03/2026"))
)

# Arrondir à la semaine
cas <- cas |>
  mutate(semaine = floor_date(date_symptomes, "week", week_start = 1))

# Courbe épidémique
ggplot(cas, aes(x = semaine)) +
  geom_histogram(binwidth = 7, fill = "steelblue", color = "white") +
  scale_x_date(date_labels = "%d/%m", date_breaks = "1 week") +
  labs(title = "Courbe epidemique",
       x = "Semaine de debut des symptomes", y = "Nombre de cas") +
  theme_classic()
```

Diagramme de flux (flowchart)

Toute étude publiée doit inclure un **diagramme de flux** (Figure 1) montrant combien de patients ont été inclus, exclus, et pourquoi. Pour Predict-R, suivre la grille **STARD** (études diagnostiques).

Structure type :

```
Patients éligibles (N = ...)
  -> Exclus (n = ...) : refus, critères non remplis
  -> Inclus (n = ...)
    -> Données complètes (n = ...)
    -> Données manquantes (n = ...)
```

En R, le diagramme de flux se fait plutôt dans Word/PowerPoint ou avec le package `DiagrammeR`. L'important est de **compter les exclusions** dans R avec `nrow()` à chaque étape de filtrage.

Grilles de lecture d'articles

- **STROBE** : études observationnelles (cohorte, cas-témoins, transversal)
- **CONSORT** : essais randomisés
- **STARD** : études diagnostiques (directement lié à Predict-R — **utiliser cette grille pour la thèse**)
- **PRISMA** : revues systématiques et méta-analyses

Vocabulaire des bases de données en santé (culture SP)

Sigle	Nom	Ce que c'est
SNDS	Système National des Données de Santé	Méga-base : SNIIRAM + PMSI + causes de décès. Couvre toute la France
SNIIRAM	Données de remboursement (Assurance Maladie)	Consultations, médicaments, actes
PMSI	Programme de Médicalisation des Systèmes d'Information	Séjours hospitaliers, GHM, diagnostics
SI-DEP	Système d'Information de DEPistage	Résultats de tests COVID-19
Géodes	Plateforme SpFrance	900+ indicateurs de santé en ligne

> Tu n'auras pas accès au SNDS tout de suite, mais **connaître le vocabulaire** est attendu en entretien ARS/SpFrance.

Veille et alerte sanitaire (vocabulaire ARS)

Terme	Définition
HCSP	Haut Conseil de la Santé Publique — émet les avis/recommandations (vaccination, dépistage)
MDO	Maladie à Déclaration Obligatoire (ex: tuberculose, rougeole, COVID) — tout médecin doit signaler
Signalement	Notification d'un cas ou cluster à l'ARS -> déclenche une investigation
Cas index	Premier cas identifié dans un épisode
Cas secondaire	Cas infecté par le cas index
Taux d'attaque	Proportion de malades parmi les exposés = cas / exposés

Terme	Définition
Contact tracing	Identifier et suivre les personnes en contact avec un cas confirmé
Investigation d'épisode	Enquête ARS : confirmer les cas, identifier la source, mesures de contrôle
MINSANTE	Message d'alerte sanitaire du ministère aux ARS
Point épidémiologique	Bulletin régulier SpFrance sur la situation d'une maladie

> **En entretien ARS** : on te demandera "décrivez le circuit de signalement d'une MDO" -> Médecin signale à l'ARS -> ARS vérifie et investigue -> SpFrance consolide au niveau national -> retour d'info aux professionnels.

G. Bases mathématiques

La loi normale (Gauss)

La distribution en cloche. La plupart des mesures biologiques la suivent (taille, poids, DFG...).

Propriétés : - Symétrique autour de la moyenne (mu) - **Règle 68-95-99.7** : 68% dans [mu-sigma, mu+sigma], 95% dans [mu-2sigma, mu+2sigma], 99.7% dans [mu-3sigma, mu+3sigma] - mu = moyenne, sigma = écart-type

Variance et écart-type

- **Variance** = moyenne des carrés des écarts à la moyenne -> mesure la dispersion
- **Écart-type (SD)** = racine carrée de la variance -> même unité que les données
- En R : `var(x)` et `sd(x)`
- Si SD = 12 pour un âge moyen de 65 -> ~68% des patients ont entre 53 et 77 ans

Le théorème central limite

"La **distribution d'échantillonnage de la moyenne** d'un échantillon suffisamment grand (n > 30) tend vers une loi normale, quelle que soit la distribution d'origine." (Autrement dit : la moyenne elle-même varie d'un échantillon à l'autre — c'est cette variation qui suit une loi normale, pas les données brutes.) - Justifie l'utilisation des tests paramétriques même sur des données pas parfaitement normales - Plus n est grand, meilleure est l'approximation - Attention : pour des distributions très asymétriques, n > 30 ne suffit pas toujours

Degrés de liberté (df)

= nombre d'informations indépendantes dans le calcul. - Pour une moyenne : df = n - 1 - Pour un Chi2 (r lignes, c colonnes) : df = (r-1)(c-1) - Pour un Chi2 de tendance : df = 1 - Moins de df -> test plus puissant pour détecter ce qu'il cherche

Intervalle de confiance — construction

Formule générale : IC 95% = estimation +/- 1.96 x erreur standard - Moyenne : x_barre +/- 1.96 x (SD / racine(n)) - OR : exp(ln(OR) +/- 1.96 x SE) - Le 1.96 = quantile 97.5% de la loi normale standard

Interprétation rigoureuse : "Si on répétait l'étude 100 fois, 95 des IC contiendraient la vraie valeur." (et non "95% de chance que la vraie valeur soit dedans")

Puissance statistique

- **Puissance (1-beta)** = probabilité de détecter un effet qui existe
- Puissance standard : 80% (on accepte 20% de risque de rater un vrai effet)
- Dépend de : taille d'échantillon, taille de l'effet, seuil alpha, variabilité

```r
# install.packages("pwr")
library(pwr)
# Puissance pour un test de proportion (Predict-R)
pwr.p.test(h = ES.h(p1 = 0.15, p2 = 0.10),   # effet attendu vs H0
           sig.level = 0.05,                     # alpha
           power = 0.80)                         # -> donne n nécessaire
```

En entretien, on te demandera : "Comment avez-vous déterminé la taille de votre échantillon ?" -> répondre avec la puissance, l'effet attendu et le seuil alpha.

Dernière mise à jour : Session 11 — 26 mars 2026 (v4 — corrections finales + analyse de sensibilité + vocabulaire ARS + messages d'erreur + Quarto + tabyl)